Spherepop

Histories, Continuations, and the Algebra

of Reachable Computation

A Complete Theory including Type Systems,

Error Correction, and Garbage Collection

Flyxion

Independent Researcher

Abstract

Most programming languages begin with values, variables, and state transitions. Spherepop begins elsewhere: with the claim that histories are ontologically prior to states, and that computation is the progressive restriction of possibility rather than the manipulation of values.

This monograph presents the complete theory of the Spherepop programming language and its philosophical foundations. We proceed from two converging arguments. The first is a theory of notes: the observation that notes are not passive records of the past but anti-collapse artifacts that preserve access to continuations. The second is the Spherepop inversion: the observation that state is a collapsed history and that taking this seriously demands a new computational ontology.

These arguments converge because they are the same argument. A note is an externalized continuation. A Spherepop history is an internalized note. Civilization builds note infrastructure; Spherepop builds note infrastructure for computation.

The monograph develops this convergence through nine parts. Part I establishes the dual failure of the passive theory of notes and the state-centric theory of computation. Part II derives the four Spherepop operators—Pop, Refuse, Bind, and Collapse—from first principles as the minimal complete set for managing possibility space. Part III develops the full operational semantics, the History category, and collapse as quotient. Part IV presents the complete admissibility type system with Progress, Preservation, and Collapse Soundness theorems. Part V derives the seven note classes as physical realizations of the four operators, establishing that notes are Spherepop programs and Spherepop programs are notes. Part VI covers compilation, the Event IR, history equivalence as compiler correctness, garbage collection as admissibility-guided collapse, and error correction as certified refusal. Part VII develops the deep mathematical structure: sheaf semantics, CLIO projections, the Santoro–Waghmare–Panaretos unification, and active geodesic inference. Part VIII situates the frame-

work within civilization-scale note infrastructure, MEM|8, and Repair Theory. Part IX collects open problems.

Four principal theorems anchor the work: the Conservation of Possibility, the Ephemerality Inversion, the Civilizational Collapse Theorem, and the History–Observation Adjunction. Four implementation discoveries retroactively sharpened the theory: refusal must carry reasons; collapse must specify quotient rules; type checking depends on the assumed boundary of the world; and compiler correctness requires history equivalence, not mere output equivalence.

Contents

Part I

The Problem

Chapter 1

The Passive Theory of Notes and Its Failure

There is a theory of notes so pervasive that it rarely receives explicit statement. On this theory, a note is a device for compensating the limitations of biological memory. Something happens—a meeting is scheduled, a formula is derived, a perception is formed—and because the mind cannot reliably retain it, the note is made. The note is therefore a prosthetic memory: it externalizes information onto a more durable substrate, where it may be retrieved when biological recall fails.

This is the passive theory of notes. It places the note in a retrospective relation to time. The note exists because something happened in the past that is worth preserving. Its value is proportional to the fidelity with which it reproduces a prior state: a good note accurately records what occurred; a poor note distorts it; an ephemeral note is low-value because it preserves little and does not endure.

The passive theory is not entirely wrong. Many notes are made for the reason it describes. But as a general account of what notes are and why they matter, it fails on its own terms and in a direction that reveals something important about cognition, language, and civilization.

The failure becomes apparent the moment one examines not celebrated notes but common ones. A grocery list does not record anything that happened. The items on the list do not exist yet in the form the list anticipates: they have not been purchased, assembled, or carried home. The list records a future act. A blueprint describes a building not yet constructed. Source code records computations not yet run. A theorem records consequences not yet derived. A calendar records commitments not yet honored. A playlist records a future act of listening. A bookmark records a future act of reading.

None of these are memories. All of them are notes.

The passive theory cannot accommodate this class of cases without becom-

ing incoherent. It might be extended to say that a note records an *intention* rather than an event, but this extension merely postpones the difficulty. The deeper problem is that the passive theory places every note in a backward-looking relation to time. Even when the recorded content is a future intention, the note is still conceived as a trace of a prior mental event. The orientation is always retrospective.

This gets the phenomenology of note-taking almost exactly wrong. When one makes a to-do list, one is not primarily recording a past intention; one is constructing a future. The note is not oriented toward what was; it is oriented toward what has not yet occurred and what might not occur without the note's assistance.

The Continuation Account

A more adequate theory begins from a distinction that theoretical computer science has found indispensable: the difference between a *state* and a *continuation*. A state is a snapshot of a system at a moment in time—a summary, a compression, a projection. A continuation is what comes next: the future execution enabled by a present configuration, the program of further development that a state makes possible.

Most theories of information focus on states. A database stores states. A photograph is described as capturing a state. A memory is treated as the retention of a past state. But what makes any stored state useful is not the state itself. It is the continuation that state makes accessible.

Consider a Git commit. The commit is not valuable because it stores bytes. It is valuable because it preserves a path through development space—a specific trajectory through the possibility space of a codebase that would otherwise require expensive reconstruction. The stored state is instrumentally useful only because it is a node in a navigable graph of continuations.

Definition 1.1 (Note)**.** Let $\mathcal{C}$ denote a space of continuations and let A be an agent. A *note* is any artifact N such that there exists a continuation $c \in \mathcal{C}$ for which

$$P_A(c, t \mid N) > P_A(c, t),$$

where $P_A(c, t)$ denotes the probability that agent A can successfully reconstruct or traverse continuation c at future time t.

This definition is deliberately broad. It immediately encompasses bookmarks,

maps, photographs, source code, language, and libraries without privileging any medium. What unifies them is the functional condition: a note increases the reachability of at least one continuation.

Definition 1.2 (Continuation Volume)**.** The *continuation volume* of a note N is

$$V_C(N) = \big|\{c \in \mathcal{C} : P_A(c, t \mid N) > P_A(c, t)\}\big|.$$

Definition 1.3 (Reachability Leverage)**.** The *reachability leverage* of a note N is

$$\Lambda(N) = \frac{\sum_c \big[C_r(c) - C_r(c \mid N)\big]}{C_{\text{create}}(N)},$$

where $C_r(c)$ is the cost of reconstructing continuation c from first principles and $C_r(c \mid N)$ is the cost given access to N.

Reachability leverage varies over many orders of magnitude. A ten-second bookmark may save hours of future search. A one-line equation may preserve centuries of accumulated derivation. The leverage analysis predicts which notes are most worth making and preserving, entirely independently of their fidelity to past states.

Notes as Anti-Collapse Artifacts

The Spherepop framework, developed in subsequent chapters, holds that histories are primary and states are their compressed residues. A state s is a projection

$$\sigma : \mathcal{H} \to S$$

that collapses many histories $h \in \mathcal{H}$ into a single observable state, producing what the framework calls the *state illusion*: the appearance that the present configuration is self-sufficient, when it is in fact the endpoint of a history whose fibers have been discarded.

A note is, in this framework, an anti-collapse artifact: an artifact that preserves selected fibers of $\mathcal{H}$ against the projection σ.

Proposition 1.4 (Notes as Anti-Collapse Artifacts)**.** *A note N is an anti-collapse artifact if it preserves selected fibers of $\mathcal{H}$ such that a future agent can partially reconstruct elements of $\mathcal{H}$ rather than being restricted to $\sigma(\mathcal{H})$.*

This is the precise sense in which the passive theory inverts the correct account. The passive theory holds that a note preserves the past against forgetting. The correct account holds that a note preserves access to futures against collapse.

The distinction is not merely philosophical: it determines what kinds of notes matter, how notes should be evaluated, and what it means for a note to succeed or fail.

Chapter 2

The State-Centric Assumption and Its Consequences

The Standard Premise

Open any introductory programming text and you encounter a first lesson: a variable is a named location that holds a value. A program begins in some initial state, instructions modify that state, and the final state is the answer.

Listing 2.1: The standard state-centric view

```
x = 5
y = x + 1
```

This appears natural. It mirrors how we naively think about physical processes: an object has properties; those properties change; the current properties constitute the object's state. But the appearance of naturalness is produced by a prior choice—the choice to treat present configuration as the fundamental unit of description.

The State Illusion

Consider what this choice erases. In version control, the repository is not merely its current file tree; it is the entire sequence of commits that produced it. In event sourcing, database records are append-only event logs; the current state is a fold over that log. In scientific experiment, a measurement is the terminus of a sequence of decisions, calibrations, and interpretive choices. In evolutionary biology, a genome is a compressed record of selective pressures.

In each case, the historical structure is epistemically and causally prior to the present configuration. State is the summary. History is the substance.

Definition 2.1 (State Degeneracy)**.** Let E be an event alphabet, $\mathcal{H} = \bigcup_{n=0}^{\infty} E^n$ the set of all finite histories, and S an observable state space. A *state projection* is

a function $\sigma : \mathcal{H} \to S$. The *degeneracy* of a state $s \in S$ is

$$D(s) = |\sigma^{-1}(s)|.$$

Proposition 2.2 (The State Illusion)**.** *For any nontrivial finite state space S and any non-injective projection $\sigma : \mathcal{H} \to S$, there exist distinct histories $H_1 \neq H_2$ such that $\sigma(H_1) = \sigma(H_2)$. Consequently,*

$$\sigma(H_1) = \sigma(H_2) \not\Rightarrow H_1 = H_2.$$

Proof. Since $|\mathcal{H}|$ is countably infinite and $|S|$ is finite, σ cannot be injective. Therefore $D(s) > 1$ for at least one $s \in S$, which yields the required $H_1 \neq H_2$ with $\sigma(H_1) = \sigma(H_2)$. □

Remark 2.3. Proposition 2.2 is the formal statement of the state illusion: any program that exposes only $\sigma(H)$ to its user has silently discarded distinguishing information about the history H. The conventional debugger is an instrument for reconstructing H from $\sigma(H)$ after information has already been lost.

Consequences for Language Design

Programming languages that treat state as primary inherit the degeneracy of Proposition 2.2 as a structural feature. Race conditions arise because σ is non-injective on interleaved histories. Use-after-free vulnerabilities arise because σ conflates histories that differ only in deallocation events. The auditing and provenance tools that practitioners add to production systems are, in each case, partial reconstructions of H from $\sigma(H)$.

The Spherepop design decision follows directly: if history is what is lost, history must be what is preserved.

The Connection to Notes

The state illusion and the passive theory of notes are the same mistake viewed from different angles.

The passive theory treats a note as a record of a past state. The state-centric theory of computation treats a program's output as its definitive product. Both identify value with a terminal state rather than with the trajectory that produced it. Both discard the history in favor of the summary.

The Spherepop framework and the continuation theory of notes are therefore the same correction viewed from different angles. Both insist that the trajectory is primary. Both insist that what matters is not what a system is at a moment but what it has done and what it can still do. Both treat state as a derived notion: useful for some purposes, but never the fundamental unit of description.

This convergence is the organizing principle of the present monograph.

Part II

First Principles

Chapter 3

Deriving the Operators

The Minimal Requirements

If histories are primary and states are derived, then a computational framework must at minimum be able to:

1. Commit to a specific continuation from among the available possibilities.
2. Document that a possible continuation is inadmissible, with reasons.
3. Record a dependency between two elements of the history.
4. Observe the current state of a history under a specific observational framework.

We claim that these four requirements are not only necessary but sufficient: they constitute the minimal complete set of operations on possibility space. All other computational phenomena are derived from these four.

Definition 3.1 (Event Alphabet)**.** Let E be a set of events, partitioned into primitive generators:

$$E = \{\mathsf{Pop}\} \cup \{\mathsf{Refuse}\} \cup \{\mathsf{Bind}\} \cup \{\mathsf{Collapse}\} \cup E_\lambda$$

where E_λ contains the lambda-calculus events $\{\mathsf{LamIntro}, \mathsf{Apply}\}$ and scope markers.

Definition 3.2 (Spherepop World)**.** A *Spherepop world* is a pair $W = (H, \Omega)$ where $H \in \mathcal{H}$ is a history (finite event sequence) and $\Omega \subseteq \Omega_0$ is the current option space (a finite set of available symbols), derived from some initial option space Ω_0 with $|\Omega_0| < \infty$.

The Four Operators

Definition 3.3 (Pop: Commitment)**.** The *commitment operator* for symbol $x \in \Omega$ is

$$P_x : (H, \Omega) \mapsto (H +\!\!+ [\mathsf{Pop}(x)],\ \Omega \setminus \{x\}).$$

P_x is undefined when $x \notin \Omega$. Pop records a commitment and removes the committed symbol from the option space, foreclosing all futures in which x takes a different value.

Definition 3.4 (Refuse: Documented Inadmissibility)**.** The *refusal operator* for symbol x with reason r is

$$R_{x,r} : (H, \Omega) \mapsto (H +\!\!+ [\mathsf{Refuse}(x, r)],\ \Omega).$$

Note that Ω is unchanged: refusal records inadmissibility without foreclosing structural availability. The reason r is a first-class component of the operation. A refusal without a reason documents nothing.

Definition 3.5 (Bind: Dependency Declaration)**.** The *bind operator* is

$$B_{a,b} : (H, \Omega) \mapsto (H +\!\!+ [\mathsf{Bind}(a, b)],\ \Omega).$$

Bind records that two trajectory elements are coupled without consuming either from the option space.

Definition 3.6 (Collapse: Observation under Rule)**.** The *collapse operator* with collapse rule c is

$$C_{x,c} : (H, \Omega) \mapsto (H +\!\!+ [\mathsf{Collapse}(x, c)],\ \Omega),$$

subject to the admissibility precondition $\Gamma \vdash t : \mathsf{Admissible}(T)$. Collapse requires an admissibility certificate. A term whose type is not $\mathsf{Admissible}(\cdot)$ cannot be collapsed—this is a hard type error, not a runtime exception.

Why These Four and No Others

The minimality of the four operators follows from an analysis of what operations on possibility space are conceptually irreducible.

Pop is irreducible because commitment—the foreclosure of alternative futures—cannot be expressed as a combination of documentation, dependency recording,

or observation. It is the primitive act of narrowing possibility.

Refuse is irreducible because documented inadmissibility is distinct from commitment: it records that a path was considered and found inadmissible, which is information that commitment cannot record. A commit of the alternative does not document why the refused path was refused. Reasons are evidence that survives collapse; without them, refusal is indistinguishable from non-consideration.

Bind is irreducible because dependency recording—coupling two trajectory elements—cannot be expressed as commitment or refusal. A bind creates a relationship between elements without consuming either; no combination of Pop and Refuse produces this effect.

Collapse is irreducible because observation under a specific rule is distinct from all three of the above. Pop, Refuse, and Bind all operate on the history and option space; Collapse maps from history space to observable state space under a specified projection. It is the meta-operation that makes histories visible.

Theorem 3.7 (Completeness of the Four Operators)**.** *The four operators* $\{P_x, R_{x,r}, B_{a,b}, C_{x,c}\}$ *are sufficient to express: variable assignment, conditional branching, function application, exception handling, memory deallocation, and concurrent event recording.*

Proof. We establish expressibility for each construct.

Variable assignment $x := v$: Express as P_v followed by $B_{x,v}$. The commitment records the value choice; the bind records the name-value dependency.

Conditional branch on predicate ϕ: Express as P_{ϕ^+} if ϕ holds, or $R_{\phi^+,\mathsf{ConstraintViolation}(\phi)}$ followed by P_{ϕ^-} if not. The refusal records why the positive branch was not taken.

Function application $f(a)$: Expressed as $\mathsf{LamIntro}(x)$ followed by $\mathsf{Apply}(a)$—a deferred Pop. The closure freezes the option space at the moment of abstraction; application is the moment of commitment.

Exception throwing: Expressed as $R_{t,r}$ (documented inadmissibility without system destruction). The type $\mathsf{Refused}(T, r)$ carries the reason as a first-class certificate.

Memory deallocation: Expressed as $R_{p,\mathsf{Deallocated}}$ where p is the pointer symbol. The refusal records that the memory region is no longer admissible to access, making use-after-free a type-level error.

Concurrent event recording: Expressed via Meld, the monoidal composition of two histories. Two concurrent worlds (H_1, Ω_1) and (H_2, Ω_2) are combined as $(H_1 +\!+ H_2, \Omega_1 \cup \Omega_2)$ under an appropriate ordering convention. □

The Conservation Law

The four operators satisfy a conservation law that reflects the fundamental structure of possibility.

Theorem 3.8 (Conservation of Possibility)**.** *For any legal execution sequence*

$$W_0 \xrightarrow{e_1} W_1 \xrightarrow{e_2} \cdots \xrightarrow{e_n} W_n$$

where each step is one of P_x, $R_{x,r}$, $B_{a,b}$, or $C_{x,c}$:

1. *$|H_t|$ is strictly monotonically increasing: $|H_{t+1}| = |H_t| + 1$.*
2. *$|\Omega_t|$ is monotonically non-increasing.*
3. *Under pure Pop dynamics, $|H_t| + |\Omega_t| = |\Omega_0|$ for all t.*

Proof. (i) Every operator appends exactly one event to H, so $|H_{t+1}| = |H_t| + 1$.

(ii) P_x removes x from Ω; the remaining operators do not modify Ω. Hence $|\Omega_{t+1}| \leq |\Omega_t|$.

(iii) Under pure Pop dynamics, every step removes exactly one element from Ω and appends one event to H. Starting from $|H_0| = 0$ and $|\Omega_0|$, after t steps $|H_t| = t$ and $|\Omega_t| = |\Omega_0| - t$, giving $|H_t| + |\Omega_t| = |\Omega_0|$. □

The conservation law has the structure of a physical conservation principle: possibility is not created or destroyed—it is either consumed (by Pop) or documented as inadmissible (by Refuse). The operators Bind and Collapse record relationships and observations without affecting the possibility budget.

Definition 3.9 (Generalised Possibility Functional)**.** Define the event weight function $w : E \to \mathbb{N}$ by $w(\mathsf{Pop}(x)) = 1$, $w(\mathsf{Refuse}(x, r)) = 0$, $w(\mathsf{Bind}(a, b)) = 0$, $w(\mathsf{Collapse}(x, c)) = 0$. For a world (H, Ω), the *generalised possibility functional* is

$$\Pi(H, \Omega) = |\Omega| + \sum_{e \in H} w(e).$$

Theorem 3.10 (Conservation of the Possibility Functional)**.** *For any legal execution sequence, $\Pi(H_t, \Omega_t) = |\Omega_0|$ for all t.*

Proof. At each step, exactly one of four operators is applied. Pop: $|\Omega|$ decreases by 1; $w(\mathsf{Pop}) = 1$ is added. Net change: $(-1) + 1 = 0$. Refuse, Bind, Collapse: $|\Omega|$ unchanged; $w = 0$ is added. Net change: 0. In every case $\Pi(H_{t+1}, \Omega_{t+1}) = \Pi(H_t, \Omega_t)$. □

Corollary 3.11 (Irreversibility of Execution)**.** *For any non-empty execution sequence, $|H_{t+1}| > |H_t|$. Therefore no legal execution can return to a previous world state (H_t, Ω_t).*

Proof. Every operator appends exactly one event to H, so $|H_{t+1}| = |H_t| + 1 > |H_t|$. Since H grows strictly, $(H_{t+1}, \Omega_{t+1}) \neq (H_t, \Omega_t)$. □

This irreversibility is not a limitation but a feature. It is the formal correlate of the phenomenological observation that histories are primary: a world that could return to a prior state would be a world in which the history of the intervening steps was undone, which is not computation but its negation.

Chapter 4

The Algebra of Histories

Histories as the Primary Objects

Before introducing types, evaluators, or compilers, we must characterize the mathematical structure of the objects that are primary in the Spherepop ontology. Histories are not merely lists of events. They form a category, and the category structure constrains what operations on histories are meaningful.

Definition 4.1 (History)**.** A *history* over event alphabet E is a finite sequence $H = [e_1, e_2, \ldots, e_n]$ with each $e_i \in E$. The empty history is $\varepsilon = []$.

Definition 4.2 (History Concatenation)**.** For histories $H_1 = [e_1, \ldots, e_m]$ and $H_2 = [f_1, \ldots, f_n]$,

$$H_1 \mathbin{+\!\!+} H_2 = [e_1, \ldots, e_m, f_1, \ldots, f_n].$$

Proposition 4.3 (History Monoid)**.** *The triple $(\mathcal{H}, +\!\!+, \varepsilon)$ is the free monoid over E.*

Proof. Associativity of $+\!\!+$ follows from list concatenation associativity. ε is the left and right identity. Freeness: every history H is uniquely determined by its event sequence, and no non-trivial relation among generators holds. □

The History Category

Definition 4.4 (History Category **Hist**)**.** Define the category **Hist** as follows:

- *Objects*: option spaces $\Omega \subseteq \Omega_0$.
- *Morphisms*: $\mathrm{Hom}(\Omega, \Omega')$ is the set of histories H such that starting from Ω, executing H yields Ω'.
- *Composition*: $H_2 \circ H_1 = H_1 \mathbin{+\!\!+} H_2$ (execute H_1 first, then H_2).
- *Identity*: $\mathrm{id}_\Omega = \varepsilon$.

Proposition 4.5 (**Hist** is a Well-Defined Category)**.** **Hist** *satisfies all category axioms.*

Proof. Associativity of composition follows from associativity of $++$. ε is the identity morphism since $H ++ \varepsilon = H = \varepsilon ++ H$. □

Proposition 4.6 (**Hist** is a Free Strict Monoidal Category)**.** $(\mathbf{Hist}, ++, \varepsilon, \otimes)$ *is a free strict monoidal category where the tensor product* $H_1 \otimes H_2$ *is the Meld operation* (*parallel composition of histories*) *and the monoidal unit is the empty option space with empty history.*

Remark 4.7. The free monoid structure forces `append` (extend by one generator) and `meld` (monoidal composition of two histories) as the only legitimate history operations. The absence of `remove` and `undo` is the mechanization of freeness: there are no relations among generators.

Listing 4.1: History as free monoid in Rust

```
pub fn append(&mut self, event: Event) {
    self.events.push(event);  // one generator
}
pub fn meld(&mut self, other: &History) {
    self.events.extend(other.events.iter().cloned());
}
```

History equality $H_1 = H_2$ is the primary criterion of program identity.

Unique Factorisation

Theorem 4.8 (Unique History Factorisation)**.** *Every non-empty history* $H = [e_1, e_2, \ldots, e_n]$ *admits a unique factorisation into primitive generators:*

$$H = [e_1] ++ [e_2] ++ \cdots ++ [e_n].$$

No other factorisation into generators exists.

Proof. Since $\mathcal{H}$ is the free monoid over E, every element has a unique normal form as a sequence of generators. The generators are singletons $[e]$; freeness of the monoid guarantees there are no non-trivial relations among generators. □

Remark 4.9. Unique factorisation is what makes history equivalence a decidable correctness criterion for compilation. If two execution paths produce the same history, they must have produced the same sequence of primitive events in the same order.

Admissibility as a Set-Valued Function

Definition 4.10 (Admissible Futures)**.** For a history H, the set of admissible future symbols is

$$\mathcal{A}(H) = \Omega_0 \setminus \{x \mid \exists r.\, \mathsf{Refuse}(x, r) \in H\}.$$

Definition 4.11 (Admissibility Contraction)**.** Let $H' = H +\!\!+ [\mathsf{Refuse}(x, r)]$. Then $\mathcal{A}(H') \subseteq \mathcal{A}(H)$, and specifically $x \notin \mathcal{A}(H')$ while x may have been in $\mathcal{A}(H)$.

Remark 4.12. The asymmetry between Pop and Refuse is critical. Pop contracts the structural option space Ω. Refuse contracts the admissibility set $\mathcal{A}(H)$ without touching Ω. Two execution paths may have the same Ω while having different $\mathcal{A}(H)$—they are structurally equivalent but semantically distinguished by their refusal histories. This distinction is invisible to any state-centric computational model.

Part III

Operational Semantics and Collapse

Chapter 5

Small-Step Semantics

The Transition System

We write $(H, \Omega) \xrightarrow{\alpha} (H', \Omega')$ for a single execution step labelled α.

$$\frac{x \in \Omega}{(H, \Omega) \xrightarrow{\mathsf{pop}(x)} (H +\!\!+ [\mathsf{Pop}(x)],\ \Omega \setminus \{x\})} \quad \text{[POP]}$$

$$\frac{}{(H, \Omega) \xrightarrow{\mathsf{refuse}(x,r)} (H +\!\!+ [\mathsf{Refuse}(x, r)],\ \Omega)} \quad \text{[REFUSE]}$$

Note: [REFUSE] has no premise—any symbol may be refused. The effect on the option space is nil; the effect on admissibility is non-nil.

$$\frac{}{(H, \Omega) \xrightarrow{\mathsf{bind}(a,b)} (H +\!\!+ [\mathsf{Bind}(a, b)],\ \Omega)} \quad \text{[BIND]}$$

$$\frac{\Gamma \vdash t : \mathsf{Admissible}(T)}{(H, \Omega) \xrightarrow{\mathsf{collapse}(x,c)} (H +\!\!+ [\mathsf{Collapse}(x, c)],\ \Omega)} \quad \text{[COLLAPSE]}$$

The premise $\Gamma \vdash t : \mathsf{Admissible}(T)$ is the type-theoretic admissibility guard. Collapse requires an admissibility certificate. Without it, collapse is a hard type error.

Big-Step Semantics

The big-step relation $\langle t, W \rangle \Downarrow \langle v, W' \rangle$ relates terms and worlds to values and resulting worlds.

Definition 5.1 (Evaluation Result)**.** An *evaluation result* is a triple (v, H, a) where v is a value, H is the accumulated history, and $a \in \{\top, \bot\}$ is the admissibility flag. The admissibility flag is $\top$ if every operation in H was admissible and $\bot$ if

any refusal was encountered.

The key invariant is that evaluation never discards history. Every step appends to H; no step modifies or removes prior events. The world grows monotonically.

Listing 5.1: Evaluation result preserving all history

```
pub struct EvalResult {
    pub value: Value,
    pub world: World,    // world.history grows monotonically
    pub admissible: bool,
}
```

Multi-Step Reduction

Definition 5.2 (Multi-Step Reduction)**.** The multi-step reduction $\to^*$ is the reflexive transitive closure of $\to$. A term t is *normal* if there is no t' with $t \to t'$.

Theorem 5.3 (History Monotonicity)**.** *If $(H_0, \Omega_0) \to^* (H_n, \Omega_n)$ then H_0 is a prefix of H_n: there exists H' such that $H_n = H_0 +\!+ H'$.*

Proof. By induction on the length of the reduction sequence. Each step appends exactly one event to the history. The result follows by transitivity. □

Theorem 5.4 (Option Space Monotonicity)**.** *If $(H_0, \Omega_0) \to^* (H_n, \Omega_n)$ then $\Omega_n \subseteq \Omega_0$.*

Proof. Only Pop removes elements from Ω; all other operators leave it unchanged. By induction, Ω can only shrink. □

Chapter 6

Collapse as Quotient

The Insufficiency of Unparameterized Collapse

The original Spherepop design represented collapse as simply `Collapse(Symbol)`. This was insufficient, and the insufficiency is philosophically illuminating.

Observable state depends on how you look. The same history, examined by different observers with different observational frameworks, yields different observable states. A collapse without a rule is a collapse into an unspecified notion of visibility. The rule is not an implementation detail; it is what makes the observation meaningful.

Definition 6.1 (Collapse Rule)**.** A *collapse rule* is a function $c : \mathcal{H} \to O_c$ from the history monoid to an observational space O_c.

Definition 6.2 (Observational Equivalence)**.** Two histories $H_1, H_2 \in \mathcal{H}$ are *observationally equivalent under rule* c, written $H_1 \sim_c H_2$, if and only if $c(H_1) = c(H_2)$.

Definition 6.3 (Observable State as Quotient)**.** The *observable state space* under rule c is the quotient:

$$O_c = \mathcal{H}/\sim_c.$$

An observable state is an equivalence class of histories.

Observable state is not a state. Observable state is a quotient. Two histories that are globally distinct may be observationally indistinguishable under a specific collapse rule. This is a feature: the collapse rule specifies which distinctions are relevant to a given question.

The Four Canonical Rules

Definition 6.4 (Identity Rule)**.** $c_I : \mathcal{H} \to \mathcal{H}$ is the identity: $c_I(H) = H$. Under the identity rule, every event is visible and $O_I \cong \mathcal{H}$.

Definition 6.5 (LastWrite Rule)**.** c_{LW} maps H to the most recent pop for each

symbol, with subsequent refusals retroactively removing symbols:

$$c_{LW}(H) = \{x \mapsto 1 \mid \mathsf{Pop}(x) \in H \text{ and } \nexists r.\, \mathsf{Refuse}(x, r) \in H \text{ after the last } \mathsf{Pop}(x)\}.$$

Definition 6.6 (Accumulate Rule)**.** $c_{\text{acc}}(H)$ maps each symbol to the count of its Pop events:

$$c_{\text{acc}}(H)(x) = |\{i \mid e_i = \mathsf{Pop}(x)\}|.$$

Definition 6.7 (Projection Rule)**.** For a label prefix L, the projection rule π_L restricts to events whose symbol names share prefix L:

$$\pi_L(H) = [e_i \in H \mid \text{sym}(e_i) \text{ starts with } L].$$

Proposition 6.8 (Coarsening Order)**.** *The collapse rules form a partial order by coarsening: $c_1 \leq c_2$ iff $\sim_{c_2} \subseteq \sim_{c_1}$ (finer equivalence = more informative observation). Under this order: c_I is the finest; π_L is coarser than c_I; c_{LW} and c_{acc} are incomparable in general.*

The Collapse Functor

Definition 6.9 (Observable-State Category $\mathbf{Obs}_c$)**.** For a collapse rule c, define the category $\mathbf{Obs}_c$:

- *Objects*: observable states $o \in O_c$.
- *Morphisms*: $\text{Hom}(o_1, o_2)$ is the set of observable transitions transforming o_1 into o_2 under rule c.
- *Composition*: sequential observable transition.

Definition 6.10 (Collapse Functor)**.** For collapse rule c, define $F_c : \mathbf{Hist} \to \mathbf{Obs}_c$ by $F_c(\Omega) = c(\varepsilon_\Omega)$ on objects and $F_c(H) = c(H)$ on morphisms.

Theorem 6.11 (Functoriality of Collapse)**.** *F_c is a well-defined functor when c respects composition:*

$$c(H_2 +\!+ H_1) = c(H_2) \circ c(H_1).$$

Proof. Identity preservation: $F_c(\varepsilon) = c(\varepsilon) = \text{id}_{O_c}$. Composition preservation: $F_c(H_2 \circ H_1) = F_c(H_1 +\!+ H_2) = c(H_1 +\!+ H_2) = c(H_2) \circ c(H_1) = F_c(H_2) \circ F_c(H_1)$ when c respects composition. □

Remark 6.12. The functoriality of collapse elevates it from an implementation detail to a category-theoretic object. Collapse is a structure-preserving map between categories. The collapse rule specifies which structures are preserved,

and different rules preserve different structures. c_{LW} is not generally functorial because the last-write of a concatenated history is not the composition of the last-writes of its parts—a later segment can retroactively affect the observable state of an earlier one.

The Universal Property of Collapse

Theorem 6.13 (Universal Property of Collapse)**.** *Let $q_c : \mathcal{H} \to O_c$ be the quotient map for rule c. If $f : \mathcal{H} \to X$ is any function satisfying*

$$H_1 \sim_c H_2 \implies f(H_1) = f(H_2),$$

then there exists a unique $\bar{f} : O_c \to X$ such that $f = \bar{f} \circ q_c$.

Proof. By the universal property of quotient sets. Define $\bar{f}([H]_c) = f(H)$. This is well-defined because f is constant on equivalence classes. Uniqueness: any $\bar{f}$ satisfying $f = \bar{f} \circ q_c$ must map $[H]_c$ to $f(H)$. □

Remark 6.14. The universal property says that observable state is not one representation among many. It is the universal representation among all representations that identify the same histories as c does. Every coarser observation necessarily passes through O_c.

Part IV

Type Theory

Chapter 7

Types as Admissibility Certificates

The Inadequacy of Value-Set Types

The standard interpretation of types—a type is a set of admissible values—is adequate for state-centric languages. It is insufficient for Spherepop.

Consider what a type must certify in a history-primary setting. A value v of type T is not merely a member of a set. It is the result of a path through history that reached v without violating any constraint represented by T. The type is not a description of v at a moment; it is a certificate about the history that produced v.

The Type Language

Definition 7.1 (Spherepop Types)**.** The type language of Spherepop is defined inductively:

$$\begin{aligned} T ::= & \ \mathsf{Unit} \mid \mathsf{Never} \mid X \\ & \mid \Pi(x : T_1).T_2 \\ & \mid \mathsf{Admissible}(T) \\ & \mid \mathsf{Refused}(T, r) \\ & \mid \mathsf{Collapsed}(T, c) \\ & \mid \mathsf{Process}(T_1 \rightsquigarrow T_2) \end{aligned}$$

The simple function type $T_1 \to T_2$ is notation for $\Pi(\_ : T_1).T_2$.

The distinctive types carry certificates, not just values:

- $\mathsf{Admissible}(T)$: the path that produced this term was admissible.
- $\mathsf{Refused}(T, r)$: the term was encountered but found inadmissible for reason r. The reason is part of the type—two terms refused for different reasons

have genuinely different types.

- $\mathsf{Collapsed}(T, c)$: an admissible term was observed under collapse rule c and the observation is sealed. The rule c is part of the type.

The Type Rules

We write $\Gamma \vdash t : T$ for the standard typing judgement.

$$\frac{\Gamma \vdash t : T}{\Gamma \vdash \mathsf{pop}(t) : \mathsf{Admissible}(T)} \quad \text{[T-POP]}$$

Pop transforms any typed term into an admissible term. It is the mechanism by which structural commitment produces a reachability certificate.

$$\frac{\Gamma \vdash t : T \quad r \in \mathsf{RefusalReason}}{\Gamma \vdash \mathsf{refuse}(t, r) : \mathsf{Refused}(T, r)} \quad \text{[T-REFUSE]}$$

Refusal is typed: the type carries the reason. A term refused for $\mathsf{ConstraintViolation}(c)$ has a type formally distinct from one refused for $\mathsf{NotAvailable}$.

$$\frac{\Gamma \vdash t : \mathsf{Admissible}(T) \quad c \in \mathsf{CollapseRule}}{\Gamma \vdash \mathsf{collapse}(t, c) : \mathsf{Collapsed}(T, c)} \quad \text{[T-COLLAPSE]}$$

Collapse requires an admissibility certificate. A term whose type is not $\mathsf{Admissible}(\cdot)$ cannot be collapsed—this is a hard type error, not a runtime exception.

$$\frac{\Gamma \vdash a : T_1 \quad \Gamma \vdash b : T_2}{\Gamma \vdash \mathsf{bind}(a, b) : \mathsf{Process}(T_1 \rightsquigarrow T_2)} \quad \text{[T-BIND]}$$

$$\frac{\Gamma, x : T_1 \vdash b : T_2}{\Gamma \vdash \lambda x : T_1.\, b : \Pi(x : T_1).T_2} \quad \text{[T-LAM]}$$

$$\frac{\Gamma \vdash f : \Pi(x : T_1).T_2 \quad \Gamma \vdash a : T_1}{\Gamma \vdash f\ a : T_2[x := a]} \quad \text{[T-APP]}$$

Open Worlds and Closed Worlds

During implementation of the Spherepop type checker, a conflict emerged between how the interpreter and type checker treated free variables. The apparent bug conceals a genuine theoretical discovery: which behavior is correct depends on which assumption about the boundary of the world is in force.

Definition 7.2 (Closed-World Context)**.** A *closed-world context* Γ_c satisfies:

$$x \notin \Gamma_c \implies \Gamma_c \vdash x : \bot.$$

The absence of a declaration is the absence of the object.

$$\frac{x \notin \Gamma \quad \Gamma \text{ is closed-world}}{\Gamma \nvdash x : T \text{ for any } T} \quad \text{[T-VAR-CLOSED]}$$

Definition 7.3 (Open-World Context)**.** An *open-world context* Γ_o satisfies:

$$x \notin \Gamma_c \implies \Gamma_o \vdash x : \mathsf{Unit}.$$

The absence of a declaration is undecided possibility.

$$\frac{x \notin \Gamma \quad \Gamma \text{ is open-world}}{\Gamma \vdash x : \mathsf{Unit}} \quad \text{[T-VAR-OPEN]}$$

Proposition 7.4 (Monotonicity of Provability)**.** $\Gamma_c \subseteq \Gamma_o \implies \text{Provable}(\Gamma_c) \subseteq \text{Provable}(\Gamma_o)$.

Remark 7.5. The open/closed world distinction is not an engineering convenience. It is the mechanization of an epistemological commitment: a closed-world type checker assumes complete knowledge of the reachable; an open-world type checker assumes partial knowledge. A language whose fundamental ontology is about possibility space cannot leave this distinction implicit. The TypeMode is preserved through `extend`: a closed-world context cannot silently become open-world mid-derivation.

Principal Theorems

Theorem 7.6 (Collapse Soundness)**.** *If* $\Gamma \vdash \mathsf{collapse}(t, c) : T$ *then* $\Gamma \vdash t : \mathsf{Admissible}(T')$ *for some* T'*, and* $T = \mathsf{Collapsed}(T', c)$.

Proof. By inversion on [T-COLLAPSE]: the only rule that derives a Collapse judgement requires the premise $\Gamma \vdash t : \mathsf{Admissible}(T)$, so the inner term must have an admissible type. □

Theorem 7.7 (Type Preservation)**.** *If* $\Gamma \vdash t : T$ *and* $t \to t'$*, then* $\Gamma \vdash t' : T$.

Proof. By structural induction on the reduction relation $\to$. The key cases are [T-POP], [T-REFUSE], and [T-COLLAPSE], each mapping a reduction step to an admissibility transformation closed under the respective rule. Beta-reduction

requires a substitution lemma: $\Gamma, x : T_1 \vdash b : T_2$ and $\Gamma \vdash a : T_1$ implies $\Gamma \vdash b[x := a] : T_2[x := a]$. □

Theorem 7.8 (Progress)**.** *If* $\vdash t : T$ (*well-typed in the empty context*) *then either* t *is a value, or there exists* t' *such that* $t \to t'$.

Proof. By structural induction on t. Variable terms are values. Lambda terms are values (closures). pop, refuse, bind, and collapse of a well-typed inner term either reduce (if the inner term reduces) or produce a value (if the inner term is a value). □

Chapter 8

Dependent Types and Admissibility Lattices

Dependent Process Types

The current $\mathsf{Process}(T_1 \rightsquigarrow T_2)$ is first-order. A dependent process type allows the output type to depend on the specific value consumed.

Definition 8.1 (Dependent Process Type)**.** A *dependent process type*

$$\Pi(x : T_1).\, \mathsf{Process}(x, f(x))$$

allows the type of the output process to depend on the value x of the input. This is the natural extension of dependent type theory into the process calculus setting.

Example 8.2. A function that reads a file and returns a proof that the file was read can be typed as:

$$\Pi(\mathit{path} : \mathsf{FilePath}).\, \mathsf{Process}(\mathit{path}, \mathsf{Admissible}(\mathsf{FileContents}(\mathit{path}))).$$

The Admissible in the output type certifies that the reading path was admissible—no access violation occurred, the file existed, and the permissions were valid.

Admissibility Lattices

The current admissibility structure is Boolean: a term is admissible or not. A more refined theory recognizes that admissibility is graded.

Definition 8.3 (Admissibility Lattice)**.** An *admissibility lattice* $(\mathcal{L}, \leq, \top, \bot, \wedge, \vee)$ is a bounded lattice where:

- $\top$ is full admissibility: the trajectory satisfies all constraints.
- $\bot$ is total inadmissibility: the trajectory violates all constraints.
- $\ell_1 \wedge \ell_2$ is the admissibility under the conjunction of constraints c_1 and c_2.

- $\ell_1 \vee \ell_2$ is the admissibility under the disjunction.

Definition 8.4 (Graded Admissibility Type)**.** $\mathsf{Admissible}_\ell(T)$ denotes admissibility at level $\ell \in \mathcal{L}$. The standard $\mathsf{Admissible}(T)$ is $\mathsf{Admissible}_\top(T)$.

Proposition 8.5 (Admissibility Lattice Typing)**.** *The typing rule for graded admissibility is:*

$$\frac{\Gamma \vdash t : T \quad \ell \in \mathcal{L}}{\Gamma \vdash \mathsf{pop}_\ell(t) : \mathsf{Admissible}_\ell(T)}$$

Collapse requires admissibility at level at least ℓ_0 for some threshold ℓ_0 determined by the collapse rule c.

Remark 8.6. Admissibility lattices connect the Spherepop type theory to the admissibility geometry program of the RSVP and CLIO frameworks. The xylomorphic criterion $\lambda < 1$ from the admissibility field is a threshold condition in a continuous admissibility lattice, where admissibility is measured by a scalar λ and collapse is only valid when λ falls below the threshold.

Proof-Carrying Refusal

The RefusalReason type is currently a vocabulary for inadmissibility. It is not yet a proof system. A fully realized Spherepop type system would require RefusalReason to be a proof term: evidence that can be checked, not merely a label.

Definition 8.7 (Proof-Carrying Refusal)**.** A *proof-carrying refusal* is a refusal of the form $\mathsf{refuse}(t, \pi)$ where π is a proof term of type $\mathsf{Inadmissible}(t, \Omega)$—a formal certificate that t is inadmissible given the current admissibility set $\mathcal{A}(H)$.

Definition 8.8 (Inadmissibility Proof System)**.** The inadmissibility propositions and their proofs are:

$$\begin{aligned}
\mathsf{AlreadyRefused}(x) &: x \in \{y \mid \exists r.\, \mathsf{Refuse}(y, r) \in H\} \\
\mathsf{ConstraintViolation}(c, x) &: c(x) > \mathsf{threshold}(c) \\
\mathsf{NotAvailable}(x) &: x \notin \Omega \\
\mathsf{DependencyFailed}(x, y) &: \mathsf{Bind}(x, y) \in H \wedge \mathsf{AlreadyRefused}(y)
\end{aligned}$$

Each proposition has a type-theoretic proof term, and $\mathsf{refuse}(t, \pi)$ is only well-typed when π proves one of these propositions for t.

Theorem 8.9 (Refusal Completeness)**.** *Every inadmissible trajectory has a proof-carrying refusal at some point. That is, if $\mathcal{A}(H) \not\ni x$ for some x that is later attempted, then there exists a proof term π of $\mathsf{Inadmissible}(x, \mathcal{A}(H))$.*

Proof. $x \notin \mathcal{A}(H)$ implies $\exists r.\, \mathsf{Refuse}(x, r) \in H$. The proof term $\pi = \mathsf{AlreadyRefused}(x)$

witnesses that x is in the refused set, which is defined as the complement of $\mathcal{A}(H)$. □

Part V

Notes as Spherepop Objects

Chapter 9

The Seven Note Classes as Operator Realizations

The four Spherepop operators—Pop, Refuse, Bind, Collapse—are abstract operations on possibility space. Notes are their physical realization in cognition, computation, language, and civilization. This chapter develops the correspondence systematically.

The Classification Principle

Definition 9.1 (Dominant Operator)**.** The *dominant operator* of a note class is the Spherepop operator that accounts for the primary function of notes in that class. Most notes mix operators, but each class has a dominant one.

Theorem 9.2 (Notes as Spherepop Programs)**.** *Every note is a Spherepop program. Every Spherepop program is a note. The correspondence is given by the following bijection between note classes and operator combinations:*

Note Class	*Dominant Operator*	*Secondary Operators*
Capture	*Refuse*	—
Prospective	*Bind*	*Pop* (*deferred*)
Repair	*Refuse* + *Pop*	—
Coordination	*Bind*	*Refuse*
Refusal	*Refuse*	— (*pure form*)
Generative	*Pop*	*Bind*
Collapse	*Collapse*	—

Chapter 10

Capture Notes

Definition 10.1 (Capture Note)**.** A *capture note* is a note whose primary function is to preserve a continuation that exists at the time of note creation but is expected to become inaccessible. In Spherepop terms, a capture note performs a Refuse operation with respect to the projection $\sigma : \mathcal{H} \to S$: it preserves history fibers that the projection would collapse.

A photograph refuses the collapse of a perceptual trajectory. An audio recording refuses the collapse of a sonic trajectory. A laboratory notebook refuses the collapse of a procedural and cognitive trajectory.

Proposition 10.2 (Capture Notes and Refuse)**.** *A capture note N performs a Refuse operation with respect to the projection $\sigma : \mathcal{H} \to S$ if and only if there exists a history fiber $h \in \sigma^{-1}(S)$ such that $P_A(c_h, t \mid N) > P_A(c_h, t)$, where c_h is the continuation associated with history h.*

The quality criterion for a capture note is not fidelity to the original state but adequacy as a reconstruction substrate. A photograph that captures the correct person but not the context may be a poor capture note even at high resolution, if the lost context is what matters for future reconstruction. The adequacy of a capture note depends on what future continuations are anticipated.

Chapter 11

Prospective Notes

Definition 11.1 (Prospective Note)**.** A *prospective note* is a note whose primary function is to preserve access to a continuation that does not yet exist at note creation time but is anticipated. In Spherepop terms, a prospective note performs a Bind operation between present agent state A_t and future continuation $c_{t'}$.

A calendar entry binds future action to present scheduling. A blueprint binds future construction to present design. A roadmap binds future development to present strategic commitment. Source code binds future computation to present specification—and is unique in that the binding is mechanically executable.

Proposition 11.2 (Prospective Notes and Temporal Binding)**.** *A prospective note N performs a Bind operation if and only if*

$$P_A(c_{t'}, t' \mid N, A_t) > P_A(c_{t'}, t' \mid A_t),$$

where $t' > t$ and the conditioning on A_t indicates the agent's present state is available in both cases.

Proposition 11.3 (Collective Reachability from Coordination)**.** *Let $A_1, \ldots, A_n$ be agents with independent continuation spaces $\mathcal{C}_1, \ldots, \mathcal{C}_n$. A coordination note N increases collective reachability if*

$$|\mathcal{C}(A_1, \ldots, A_n \mid N)| > |\mathcal{C}_1| + \cdots + |\mathcal{C}_n|.$$

The proposition captures the supraadditive character of coordination: bound agents can traverse paths that no individual could reach alone.

Chapter 12

Repair, Refusal, and Generative Notes

Repair Notes

Definition 12.1 (Repair Note)**.** A *repair note* is a note whose primary function is to reduce the cost of reconstructing a continuation after it has been or is anticipated to be interrupted. It combines Refuse (preserving the history of the trajectory against collapse) and Pop (enabling re-entry at a specific point after interruption).

Theorem 12.2 (Notes as Pre-emptive Repair)**.** *Every note N with positive repair value $R_v(N, c) = C_r(c) - C_r(c \mid N) > 0$ constitutes a pre-emptive repair: it reduces expected reconstruction cost prior to any failure occurring.*

Proof. $R_v(N, c) > 0$ implies $C_r(c \mid N) < C_r(c)$. Since the note exists before any failure occurs, the cost reduction is in force from the moment of note creation. The note therefore functions as a repair mechanism that precedes the event it repairs for. □

A troubleshooting guide is a repair note: a stored Pop operator for a class of failure states. Each entry says: if you arrive at this failure state, the continuation from here to a functional state runs through these steps.

Refusal Notes

Definition 12.3 (Refusal Note)**.** A *refusal note* is a note whose primary function is to prevent the collapse of an existing distinction, independently of whether the preserved distinction is expected to be used. It is the closest realization of the pure Refuse operation.

Mathematical proofs, cryptographic checksums, archival copies, legal records, and formal specifications belong to this class.

Proposition 12.4 (Proofs as Refusal Notes)**.** *A mathematical proof P of theorem T from premises Γ satisfies*

$$P_A(c_{\Gamma\vdash T}, t \mid P) > P_A(c_{\Gamma\vdash T}, t)$$

for any agent A with sufficient background knowledge, at any future time t.

The proof refuses the collapse of an inferential trajectory into mere assertion. Its value is not actualized use but preserved potential: it maintains the reachability of the derivation whether or not anyone consults it.

Generative Notes

Definition 12.5 (Generative Note)**.** A *generative note* is a note that creates access to continuations that were not reachable before the note's creation. It is the primary realization of the Pop operation: it opens new trajectories in possibility space.

Proposition 12.6 (Generative Notes and Novel Reachability)**.** *A generative note N satisfies*

$$\exists\, c \in \mathcal{C} : P_A(c, t \mid N) > 0 \text{ and } P_A(c, t) = 0.$$

That is, N creates access to continuations that were previously unreachable.

Scientific theories, maps, programming languages, mathematical formalisms, and ontologies are generative notes. A programming language is a generative note that is mechanically executable: it defines not just a space of possible programs but a machine for traversing that space.

Chapter 13

Collapse Notes and the Ephemerality Inversion

The Hidden Class

Definition 13.1 (Collapse Note)**.** A *collapse note* is an artifact that performs a deliberate projection $\sigma_N : \mathcal{H} \to S_N$, compressing a history or artifact into a more compact representation at the cost of detail, in order to make the representation navigable. It is the realization of the Collapse operator.

File names, titles, abstracts, indexes, summaries, taxonomies, classifications, labels, and keywords are all collapse notes. The paradox of collapse notes is that they appear to be the opposite of notes—they destroy information—yet they are indispensable to the function of every other note class.

Theorem 13.2 (Necessity of Collapse Notes)**.** *For any sufficiently large collection of notes $\mathcal{N}$, there exists a threshold $|\mathcal{N}| > K$ above which the reachability of individual notes in $\mathcal{N}$ decreases in the absence of collapse notes over $\mathcal{N}$.*

Proof. Search cost through an unstructured collection grows at least linearly with collection size. For $|\mathcal{N}| > K$, exhaustive search becomes too expensive for practical use. A collapse note—index, catalogue, taxonomy—reduces search cost by imposing structure, making individual notes reachable at sublinear cost. Without such structure, notes become inaccessible through navigational failure, not destruction. □

Remark 13.3. A word is a collapse note: it compresses a complex of experience, association, inference, and usage into a single retrievable token. A noun is a collapse note about a process. The passive theory would not recognize these as notes at all. The Spherepop theory recognizes them immediately as the most pervasive and powerful class of collapse notes civilization has developed.

The Ephemerality Inversion

Definition 13.4 (Continuation Completion)**.** A continuation c is *complete* with respect to note N at time t if the trajectory encoded by N has been successfully traversed.

Theorem 13.5 (Ephemerality Inversion)**.** *Let N be a note encoding a single continuation c. If c has been completed, then the destruction of N does not reduce the reachability of any remaining continuation. Therefore the ephemerality of N after completion of c implies neither loss nor failure.*

Proof. By assumption, c is the sole continuation for which N increases reachability. After completion of c, $V_C(N) = 0$. Since continuation volume is zero, the destruction of N reduces reachability by zero. □

The grocery list discarded at the supermarket exit has not failed. It has succeeded completely. The theorem generates an inversion of the passive theory's ranking:

Fate	Passive Theory	Continuation Theory
Executed and discarded	Low value	Optimal
Preserved and re-entered	Medium value	Valuable
Preserved, never re-entered	High value	Unrealized potential

The celebrated survivors of antiquity—incomplete manuscripts, undelivered letters, abandoned plans—survive precisely because their continuations were never completed. The completed notes, the sent letters, the finished buildings, left no trace because they succeeded.

Part VI

Compilation, Garbage Collection, and Error Correction

Chapter 14

The Event IR and Compiler Correctness

The Failure of Traditional Compiler Correctness

The standard notion of compiler correctness is: a compiler is correct if the compiled program produces the same output as the interpreted program for every input. This criterion is adequate for state-centric languages. It is insufficient for Spherepop.

Two programs that produce the same final value may have constructed radically different histories to get there, and those histories are part of what the program is.

Definition 14.1 (History Equivalence as Correctness)**.** Let $I : P \to \mathcal{H}$ be the interpreter function mapping programs to histories, and $V : \mathsf{IR} \to \mathcal{H}$ the VM mapping event-IR blocks to histories, and $C : P \to \mathsf{IR}$ the compiler. A compiler C is correct if for all programs p and initial worlds W:

$$I(p, W).\mathsf{history} = V(C(p), W).\mathsf{history}.$$

Output equivalence (same final value) is a strictly weaker criterion that is insufficient for Spherepop.

Event IR

Conventional intermediate representations encode instructions that mutate machine state. Spherepop IR encodes *events*—records of occurrences that persist as part of the history the VM is constructing.

Definition 14.2 (Event IR)**.** The Spherepop intermediate representation is a sequence of IrEvent terms:

$$\mathsf{IR} ::= \mathsf{Pop} \mid \mathsf{Refuse}(r) \mid \mathsf{Collapse}(c) \mid \mathsf{Bind} \mid \mathsf{Load}(x) \mid \mathsf{Apply} \mid \cdots$$

Each event carries its reason or rule as a first-class argument, preserving the full certificate structure of the source term.

Listing 14.1: Event IR in Rust

```
pub enum IrEvent {
    Pop,
    Refuse { reason: RefusalReason },
    Collapse { rule: CollapseRule },
    Bind,
    Load(Symbol),
    Apply,
    LamIntro(Symbol, Type),
}
```

Proposition 14.3 (IR Faithfulness)**.** *The compilation function* $C : \mathsf{Term} \to \mathsf{IR}$ *is faithful: for every primitive term* t*,* $C(t)$ *contains exactly the event variants that* $I(t)$ *would append to the history, with identical arguments.*

Theorem 14.4 (Replay Equivalence)**.** *The Spherepop compiler satisfies history equivalence for all programs expressible in the Spherepop core calculus.*

Proof. By structural induction on program terms. For each primitive operation (Refuse, Bind, Collapse, Pop), the compiler emits the corresponding IrEvent variant, and the VM step performs exactly the same World mutation as the interpreter's eval case. The history appended is therefore identical. For Seq: each sub-term is compiled in order, and since histories compose by concatenation, the composed history of the compiled form equals that of the interpreted form by induction. □

Theorem 14.5 (Compiler Full Faithfulness)**.**

$$I(p_1).\mathsf{history} = I(p_2).\mathsf{history} \iff V(C(p_1)).\mathsf{history} = V(C(p_2)).\mathsf{history}.$$

Proof. ($\Rightarrow$) By Replay Equivalence applied twice. ($\Leftarrow$) Similarly. The biconditional follows from the symmetry of history equivalence. □

Chapter 15

Garbage Collection as Admissibility-Guided Collapse

The Standard Problem

In conventional languages, garbage collection identifies and reclaims memory that is no longer reachable from the program's roots. The criterion is purely structural: an object is garbage if no live reference points to it.

In Spherepop, the analogous question is: which portions of the history are no longer needed to support any reachable continuation? The answer is richer because it depends not just on structural reachability but on admissibility.

History Segments and Liveness

Definition 15.1 (History Segment Liveness)**.** A history segment $H[i : j] = [e_i, e_{i+1}, \ldots, e_j]$ is *live at time* t if there exists an agent A and a continuation c such that:

1. c is reachable at time t: $P_A(c, t) > 0$, and
2. c depends on $H[i : j]$: $P_A(c, t \mid \neg H[i : j]) < P_A(c, t)$.

A segment is *dead* if it is not live.

Definition 15.2 (Admissibility-Guided GC)**.** *Admissibility-guided garbage collection* removes dead history segments while preserving the admissibility certificates needed for future collapses. Formally, let $\mathcal{A}_{\text{live}}(H)$ be the set of future continuations still reachable. GC produces $H' \subseteq H$ (as a subsequence) satisfying:

$$\mathcal{A}(H') = \mathcal{A}(H) \cap \{x : \exists c \in \mathcal{A}_{\text{live}}(H),\ x \text{ needed by } c\}.$$

GC as Collapse

Theorem 15.3 (GC as Collapse Composition)**.** *Admissibility-guided GC is equivalent to applying a collapse rule c_{GC} that maps H to the subsequence of live events:*

$$c_{\text{GC}}(H) = [e \in H : e \textit{ is live in } H].$$

c_{GC} is a valid collapse rule when liveness is stable under extension: if e is live in H, it is live in $H +\!+ H'$ for all H'.

Proof. Liveness is defined in terms of reachable continuations. Since histories grow monotonically and Pop operations only add to the committed history, a past commitment that enables a reachable continuation remains enabling under history extension. Therefore c_{GC} respects composition: the GC of a concatenated history equals the concatenation of the GC of the parts. □

Corollary 15.4 (GC Preserves Admissibility)**.** *If $c_{\text{GC}}(H) = H'$, then $\mathcal{A}(H') \supseteq \mathcal{A}_{\text{live}}(H)$.*

Proof. GC removes only dead events. A dead Refuse event is one that marks a symbol x as inadmissible when x is no longer needed by any reachable continuation. Removing it restores x to the admissibility set. Since no live continuation needs x, this does not affect any future collapse certificate. □

Three GC Strategies

Different collapse rules for GC yield different strategies with different cost-benefit profiles.

Definition 15.5 (Mark-Scan GC)**.** *Mark-Scan GC* applies the identity rule c_I to the subhistory of live events: it retains the full sequence of live events in order. This is the most informative strategy and preserves all historical structure needed for replay.

Definition 15.6 (Quotient GC)**.** *Quotient GC* applies a coarser rule c_{LW} to the history before collecting: it retains only the last commitment to each symbol. This is equivalent to state-based GC with a history log and loses intermediate state information.

Definition 15.7 (Certificate-Only GC)**.** *Certificate-Only GC* retains only Refuse events (inadmissibility certificates) and Collapse events (observation certificates), discarding all Pop and Bind events whose certificates have been sealed. This

minimizes memory use while preserving the admissibility structure needed for future type checking.

Theorem 15.8 (GC Strategy Tradeoff)**.** *Let $M(c_{\text{GC}})$ denote memory usage and $R(c_{\text{GC}})$ denote recovery power (the set of continuations that can be reconstructed after GC). Then:*

$$M(c_I) \geq M(c_{LW}) \geq M(c_{\text{cert}}) \quad \textit{and} \quad R(c_I) \geq R(c_{LW}) \geq R(c_{\text{cert}}).$$

Proof. The identity rule retains the most events and therefore uses the most memory but also preserves the most recovery information. Each coarsening reduces both memory usage and recovery power by identifying more history segments as equivalent. □

Admissibility Certificates and GC Safety

A key constraint on GC in Spherepop is that it must not invalidate pending type judgements. If a collapse term $\mathsf{collapse}(t, c)$ is waiting on the admissibility certificate $\Gamma \vdash t : \mathsf{Admissible}(T)$, GC must retain the history events that support that certificate.

Definition 15.9 (GC Safety)**.** A GC operation is *safe* if for every pending collapse judgement $\Gamma \vdash \mathsf{collapse}(t, c) : \mathsf{Collapsed}(T, c)$, the history events needed to support $\Gamma \vdash t : \mathsf{Admissible}(T)$ are retained after GC.

Theorem 15.10 (Safety of Certificate-Only GC)**.** *Certificate-Only GC is safe: it retains all Refuse events that contribute to any pending admissibility certificate.*

Proof. The admissibility type $\mathsf{Admissible}(T)$ is established by the sequence of Pop and Refuse events that produced t. Certificate-Only GC retains all Refuse events (inadmissibility certificates) unconditionally. The Pop events can be inferred from the Collapse certificates that seal their results. Therefore all information needed for pending collapse judgements is preserved. □

Chapter 16

Error Correction as Certified Refusal

The Standard Approach and Its Limits

In conventional languages, error handling is an afterthought: exceptions are thrown when something goes wrong, and error handling code is written separately from the main computation. This approach has three problems. First, it treats errors as exceptional when they are often structurally inevitable. Second, it provides no information about why the error occurred. Third, it makes error recovery expensive because the history of the computation is not available.

In Spherepop, errors are not exceptional. Every Refuse event is a documented non-traversal of a possible path. Error handling is not separate from the main computation; it is part of the computation's history.

Error Correction Codes in History Space

Definition 16.1 (History Redundancy)**.** A history H has *redundancy factor* k if there exist k disjoint subsequences $H_1, \ldots, H_k \subseteq H$ (as event subsequences) such that each H_i alone is sufficient to reconstruct the full admissibility certificate for the terminal collapse:

$$\mathcal{A}(H_i) \supseteq \mathcal{A}_{\text{needed}}(H) \quad \text{for each } i.$$

Definition 16.2 (Erasure-Correcting History)**.** An *erasure-correcting history* with parameters (n, k) encodes k independent certificate paths through n events, such that any k of the n events are sufficient to recover the full admissibility structure.

Theorem 16.3 (Error Correction via Redundant Refusal)**.** *If a history H with redundancy factor $k \geq 2$ has $m < k/2$ events corrupted or erased, the correct admissibility structure can be recovered from the remaining events.*

Proof. With redundancy k and fewer than $k/2$ erasures, at least $k - k/2 > k/2$ redundant paths remain intact. Majority vote over the surviving paths recovers the correct admissibility judgement. □

Typed Error Propagation

Definition 16.4 (Error Type)**.** An *error type* is a type of the form $\mathsf{Refused}(T, r)$ where r is a proof-carrying reason. Error types carry the evidence of inadmissibility as first-class type-level information.

Definition 16.5 (Error Propagation Rule)**.**

$$\frac{\Gamma \vdash t : \mathsf{Refused}(T_1, r) \quad \Gamma, x : T_1 \vdash b : T_2}{\Gamma \vdash \mathsf{handle}(t, x.b) : T_2 \vee \mathbf{??}T_2, r)} \quad \text{[T-HANDLE]}$$

The handler b may produce a value of T_2 or propagate a refusal. The propagated refusal carries the original reason r, preserving the full provenance chain.

Theorem 16.6 (Error Preservation)**.** *If $\Gamma \vdash t : \mathsf{Refused}(T, r)$ and there is no handler in scope for r, then r propagates to the nearest enclosing context that can handle $\mathbf{??}\cdot, r)$.*

Proof. By the typing rules, $\mathsf{Refused}(T, r)$ types only propagate upward through [T-HANDLE] if the handler does not fully resolve them. Since every unresolved $\mathsf{Refused}(T, r)$ in a well-typed program must eventually reach a handler or become the return type of the program, propagation terminates at the program boundary with a documented reason. □

Recovery as Certified Continuation

Definition 16.7 (Recovery Certificate)**.** A *recovery certificate* for an error $\mathsf{Refused}(T, r)$ at world state W is a term $\mathsf{recover} : \mathsf{Refused}(T, r) \to \mathsf{Admissible}(T')$ that transforms a documented inadmissibility into an admissible continuation of type T'.

Theorem 16.8 (Soundness of Recovery)**.** *If $\mathsf{recover} : \mathsf{Refused}(T, r) \to \mathsf{Admissible}(T')$ is a recovery certificate and $\Gamma \vdash t : \mathsf{Refused}(T, r)$, then $\Gamma \vdash \mathsf{recover}(t) : \mathsf{Admissible}(T')$. Furthermore, the history of $\mathsf{recover}(t)$ contains the full provenance chain from the original refusal to the recovery, ensuring that the recovery is not a silent erasure of the error.*

Proof. By application of the recovery function type and [T-POP] applied to the result. The history contains the original $\mathsf{Refuse}(x, r)$ event followed by the recovery events. No history erasure is possible since histories grow monotonically. □

Remark 16.9. Recovery in Spherepop is the computational analogue of repair theory: it restores access to a continuation after documented inadmissibility, but unlike silent error suppression, it preserves the full record of what went wrong and how it was addressed. The recovery certificate is a note—specifically a Repair Note—in the formal sense of the note taxonomy.

Part VII

Deep Mathematical Structure

Chapter 17

The Observation Limit

The Reviewer's Challenge

Spherepop does not abolish the state illusion. It makes the quotient explicit, parameterised, and reversible whenever the underlying history is retained.

A careful reader will notice a tension. The document claims to resolve the state illusion—the loss of historical information caused by projecting history to state. Yet every output of a Spherepop program is a collapse: a value derived by applying a rule to a history. The programmer observes the collapsed value, not the history. Is Spherepop not simply relocating the state illusion?

This chapter addresses that challenge. We prove the No Direct Observation Theorem, which shows the challenge identifies a mathematical necessity, and then distinguish precisely between state erasure (which Spherepop eliminates) and state illusion (which no computational system can eliminate).

The No Direct Observation Theorem

Definition 17.1 (History-Faithful Observation)**.** An observation map $\phi : \mathcal{H} \to O$ is *history-faithful* if it is injective: $\phi(H_1) = \phi(H_2) \Rightarrow H_1 = H_2$.

Theorem 17.2 (No Direct Observation)**.** *Let O be any set with $|O| < |\mathcal{H}|$. Then every observation map $\phi : \mathcal{H} \to O$ is non-injective: there exist distinct $H_1 \neq H_2$ with $\phi(H_1) = \phi(H_2)$. Consequently, no computational system with a finite observable space can directly verify the history it constructed.*

Proof. Since $\mathcal{H} = \bigcup_{n \geq 0} E^n$ is countably infinite and $|O| < |\mathcal{H}|$, by the pigeonhole principle ϕ cannot be injective. Therefore there exist $H_1 \neq H_2$ with $\phi(H_1) = \phi(H_2)$. □

Remark 17.3. Theorem 7.1 is a fundamental limit. It says: every observation is a quotient. This is not a contingent limitation that better engineering can over-

come. It is a cardinality constraint. A system that made its observation map injective—that recorded every distinction in full history space—would itself be a history.

Erasure versus Illusion

Definition 17.4 (State Erasure)**.** A computational system suffers *state erasure* if it irrecoverably discards the history H after computing $\sigma(H)$. After execution, H is unavailable and only $\sigma(H)$ remains.

Definition 17.5 (State Illusion)**.** A computational system presents a *state illusion* to its programmers if they can only observe $\sigma(H)$ and not H directly. By Theorem 7.1, some state illusion is unavoidable for any non-trivial observable space.

Proposition 17.6 (What Spherepop Eliminates)**.** *Spherepop eliminates state erasure: the history H is retained in* `world.history` *and is available to any observer. Spherepop cannot and does not eliminate the state illusion: the programmer's primary observable is $c(H)$, not H, so distinct histories that agree under c remain indistinguishable from the perspective of that observation.*

The correct claim is therefore: Spherepop does not abolish the state illusion. It makes the quotient explicit, parameterised, and history-preserving.

Rule Refinement as Disambiguation

Definition 17.7 (Rule Refinement Order)**.** Collapse rule c_1 is *finer than* c_2, written $c_1 \preceq c_2$, if $H_1 \sim_{c_1} H_2 \Rightarrow H_1 \sim_{c_2} H_2$. The Identity rule c_I is the finest rule; the trivial rule $c_\top$ mapping all histories to a single point is the coarsest.

Proposition 17.8 (Disambiguation by Refinement)**.** *Let $H_1 \sim_c H_2$ (histories indistinguishable under rule c) and $H_1 \neq H_2$. Then there exists a finer rule $c' \preceq c$ such that $c'(H_1) \neq c'(H_2)$. In the limit, c_I always disambiguates since $c_I(H) = H$ exactly.*

Proof. Take $c' = c_I$: since $H_1 \neq H_2$ and $c_I(H) = H$, we have $c_I(H_1) = H_1 \neq H_2 = c_I(H_2)$. □

Chapter 18

Category Theory as Engine

The History-Observation Adjunction

Definition 18.1 (History Construction Functor)**.** Let **Terms** be the category whose objects are Spherepop terms and whose morphisms are reduction sequences. Define G : **Terms** $\to$ **Hist** by $G(t) = I(t).\mathsf{history}$.
Definition 18.2 (Observation Functor)**.** For a fixed collapse rule c, define $\mathcal{O}_c$: **Hist** $\to$ **Obs**$_c$ by $\mathcal{O}_c(H) = c(H) = F_c(H)$.
Theorem 18.3 (History-Observation Adjunction)**.** *For the Identity rule c_I, there is a natural bijection*

$$\textbf{Hist}(G(t), H) \cong \textbf{Terms}(t, G^{-1}(H))$$

whenever $G^{-1}(H)$ is defined. This is the categorical statement of Replay Equivalence: the interpreter and compiler produce histories that are morphisms in **Hist***.*

Collapse Rules Form a Lattice

Proposition 18.4 (Observational Lattice)**.** *The set of collapse rules ordered by refinement $c_1 \preceq c_2$ forms a complete lattice. The meet of $\{c_i\}$ is the finest rule at least as fine as all; the join is the rule generated by the union of equivalence relations.*

Proof. The meet and join are defined by the standard lattice operations on equivalence relations. Completeness follows because arbitrary intersections and generated equivalence relations of equivalence relations are again equivalence relations. □

Sheaf-Theoretic Semantics

Definition 18.5 (Observational Site)**.** Fix collapse rules $\{c_i\}$ and observable states $\{o_i\}$. The *observational site* $\mathcal{O}$ is the category whose objects are observational regions $\{c_i^{-1}(o_i)\}$ and whose morphisms are inclusions $c_j^{-1}(o_j) \hookrightarrow c_i^{-1}(o_i)$ when the former is contained in the latter.

Proposition 18.6 ($\mathcal{H}$ is a Sheaf)**.** *The history presheaf $\mathcal{H}$ on $\mathcal{O}$—defined by $\mathcal{H}(U) = \{H \in \mathcal{H} : H \in U\}$—satisfies the sheaf condition: compatible local histories glue uniquely to global histories.*

Proof. Compatibility means event sequences agree on their shared prefixes. Two compatible event sequences agreeing on all overlaps determine a unique concatenation. □

Proposition 18.7 (State Illusion as Non-Injectivity of Sections)**.** *A collapse rule c exhibits state illusion if and only if the observable sheaf $\mathcal{O}_c$ fails to determine a unique global section of $\mathcal{H}$. There exist distinct $H_1, H_2 \in \mathcal{H}(\mathcal{H})$ with the same section in $\mathcal{O}_c(\mathcal{H})$.*

The sheaf structure ties the local (what one observer sees under one rule) to the global (the full history that all rules project from). Sheaf cohomology measures the obstruction to lifting local observations to global histories—which is precisely the observational entropy.

Chapter 19

CLIO Projections and Active Geodesic Inference

Collapse Rules as CLIO Projections

Definition 19.1 (CLIO Projection)**.** A CLIO projection $\pi : X \to M$ maps a representation space X to a manifold M of observational strata. In the Spherepop setting: $X = \mathcal{H}$ (history space), $M = O_c$ (observable state space under rule c), and $\pi = F_c$ (the collapse functor).

Definition 19.2 (Observational Entropy)**.** For a collapse rule c and observable state $o \in O_c$, the *observational entropy* is

$$S_c(o) = \log |c^{-1}(o)|.$$

Theorem 19.3 (Entropy Monotonicity Under Rule Refinement)**.** *If $c_1 \preceq c_2$ (c_1 finer), then for every $o \in O_{c_1}$:*

$$S_{c_1}(o) \leq S_{c_2}(c_2(c_1^{-1}(o))).$$

Finer rules have smaller or equal fibre sizes.

Proof. Since $c_1 \preceq c_2$, every $\sim_{c_1}$-class is contained in a $\sim_{c_2}$-class. Therefore $|c_1^{-1}(o)| \leq |c_2^{-1}(c_2(H))|$ for any $H \in c_1^{-1}(o)$. □

Remark 19.4. Observational entropy $S_c(o) = \log |c^{-1}(o)|$ is exactly the CLIO quantity $S_\pi(m) = \log \mathrm{Vol}(\pi^{-1}(m))$ in the discrete Spherepop setting. Spherepop is a discrete computational instance of the CLIO projection program.

The Santoro–Waghmare–Panaretos Unification

Theorem 19.5 (Representation Changes Topology)**.** *Let $\rho_1, \rho_2 : X \to Y$ be two representation maps with $\tau_{\rho_1} \neq \tau_{\rho_2}$. Then there exist $x_1, x_2 \in X$ such that $x_1 \sim_{\rho_1} x_2$*

but $x_1 \not\sim_{\rho_2} x_2$, or vice versa. The two representations disagree on whether x_1 and x_2 are the same object.

Proof. If $\tau_{\rho_1} \neq \tau_{\rho_2}$, there exists an open set $U \in \tau_{\rho_1} \setminus \tau_{\rho_2}$. Then $U = \rho_1^{-1}(V)$ for some $V \subseteq Y$, but $U \neq \rho_2^{-1}(W)$ for any W. Therefore there exist x_1, x_2 with $\rho_1(x_1) = \rho_1(x_2)$ but $\rho_2(x_1) \neq \rho_2(x_2)$. □

Remark 19.6. This theorem is the general form of the observation that representations reorganise topology. It applies uniformly to: (1) the Santoro–Waghmare–Panaretos kernel embedding, which converts metrically close distributions into topologically singular Gaussian measures; and (2) Spherepop collapse rules, where refinement moves history pairs between equivalence classes.

In the kernel case, the topological change is forced by the Feldman–Hájek theorem. In the collapse case, the topological change is chosen: the programmer selects the rule, thereby choosing which distinctions are topologically visible. Both are instances of the same structural operation.

Active Geodesic Inference

The Spherepop framework connects to a broader theory of intelligent systems as trajectory-maintaining structures.

Definition 19.7 (Active Geodesic Inference)**.** An intelligent system performs *active geodesic inference* if it actively shapes its configuration space through energetic and entropic constraints, forming low-action trajectories through possibility space rather than following predetermined state transitions.

Proposition 19.8 (Spherepop as Active Geodesic Substrate)**.** *Spherepop's irreversible, scope-based semantics enforce history-sensitivity and prevent incoherent superposition by construction. Pop operations commit to specific trajectories; Refuse operations document inadmissible branches; the history preserves the full record of path selection. Spherepop is therefore a natural substrate for implementing active geodesic inference: the history is the geodesic; the admissibility set is the constraint field; collapse is the observation that seals a trajectory as complete.*

Definition 19.9 (Formal Intelligence)**.** Intelligence is the capacity of a system to remain within a family of dynamically admissible, low-action histories by actively reshaping its configuration space's geometry. This definition extends across scales, encompassing learning within a lifetime, evolution across generations, and reasoning within an episode.

Theorem 19.10 (Intelligence as Note Infrastructure)**.** *An intelligent system with formal intelligence in the above sense is necessarily a note-producing system: it constructs and maintains Capture, Prospective, and Repair notes as part of the mechanism by which it reshapes its configuration space. Notes are the mechanism by which intelligent systems install future reachability from present positions.*

Proof. A system that maintains low-action histories across time must preserve information about its past trajectories (Capture Notes), commit to future trajectories (Prospective Notes), and recover from trajectory interruptions (Repair Notes). Each of these is a note class in the formal taxonomy. Therefore any formally intelligent system produces notes as a structural necessity, not a contingent feature. □

Part VIII

Civilization as Note Infrastructure

Chapter 20

MEM|8 and the Cognitive Substrate

Memory as Event Structure

The MEM|8 architecture treats memory not as isolated symbol storage but as structured pattern access where retrieval depends on preserving relationships, locality, and accessibility. Memory is not a database; it is a cognitive substrate whose operations are structurally similar to Spherepop's history operations.

Proposition 20.1 (MEM|8 as Spherepop Instance)**.** *The MEM|8 architecture is a biological instance of the Spherepop framework:*

MEM|8 Layer	*Spherepop Note Class*
Episodic layer	*Capture notes*
Procedural layer	*Repair notes*
Semantic layer	*Coordination notes*
Indexical layer	*Collapse notes*
Theoretical layer	*Generative notes*

Remark 20.2. A notebook is a miniature MEM|8. A Git repository is a miniature MEM|8. An archive is a miniature MEM|8. Each stores structured trajectories organized to make reconstruction possible. The difference between a good archive and a poor one is not quantity of material stored but quality of access structure: how well the collapse notes and coordination notes that organize the collection preserve the reachability of trajectories within it.

Chapter 21

Repair Theory and Civilizational Collapse

Repair as Pre-emptive Refusal

Repair Theory holds that repair is the restoration of access to a trajectory that has been interrupted. The connection to Spherepop is not metaphorical but structural: repair is the execution of a Repair Note that was created in anticipation of the failure.

Theorem 21.1 (Note Taxonomy as Failure Taxonomy)**.** *The seven note classes correspond bijectively to seven classes of anticipated trajectory failure:*

Note Class	*Anticipated Failure*
Capture	*Experiential collapse*
Prospective	*Intention-execution gap*
Repair	*Procedural interruption*
Coordination	*Coordination failure*
Refusal	*Distinction collapse*
Generative	*Inferential poverty*
Collapse	*Navigational failure*

Civilizational Scale

Definition 21.2 (Civilizational Note Infrastructure)**.** Let civilization Z possess note infrastructure $\mathcal{N} = \{N_1, \ldots, N_k\}$. The *collective reachability* is

$$R(Z) = \bigcup_{i=1}^{k} \mathcal{C}(N_i),$$

where $\mathcal{C}(N_i)$ is the set of continuations preserved or enabled by N_i.

Theorem 21.3 (Civilizational Collapse)**.** *The destruction of note infrastructure $\mathcal{N}$*

decreases collective reachability even when underlying truths remain unchanged:

$$R(Z \setminus \mathcal{N}) < R(Z).$$

A civilization that loses its note infrastructure does not become ignorant. It becomes unable to reach its own knowledge.

Proof. By definition, $R(Z)$ includes all continuations preserved by elements of $\mathcal{N}$. If $\mathcal{N}$ is destroyed, no element contributes to collective reachability. Since some continuations are accessible only through $\mathcal{N}$, it follows that $R(Z \setminus \mathcal{N}) < R(Z)$. The underlying facts are not destroyed, but the paths that made them accessible are. □

Technology	Reachability Horizon	Dominant Note Classes
Speech	Minutes to hours	Coordination, Capture
Story	Decades	Capture, Generative
Writing	Centuries	All classes
Printing	Populations	Coordination, Refusal
Digital storage	Replication	All classes, at scale
Version control	Modification history	Repair, Capture
Network	Distribution	Coordination, Generative

Each note technology expands the reachability horizon by addressing failure modes the preceding technology could not prevent. Writing prevents the death of witnesses. Printing prevents the destruction of individual copies. Version control prevents the loss of developmental history through modification. The history of civilization is the history of expanding note infrastructure.

Part IX

Open Problems and Future Directions

Chapter 22

The Inverse Problem and Observational Completeness

When Does Observational Equivalence Imply History Equivalence?

The deepest open question in the Spherepop framework is the converse of the collapse functor: when does $c(H_1) = c(H_2)$ imply $H_1 = H_2$?

Definition 22.1 (Observational Completeness)**.** A collapse rule c is *observationally complete* if it is injective: $c(H_1) = c(H_2) \Rightarrow H_1 = H_2$.

Definition 22.2 (Collapse Kernel)**.** The *kernel* of a collapse rule c is

$$\ker(c) = \{(H_1, H_2) \in \mathcal{H} \times \mathcal{H} \mid c(H_1) = c(H_2)\}.$$

Observational completeness holds iff $\ker(c) = \{(H, H) \mid H \in \mathcal{H}\}$.

Proposition 22.3 (Completeness Results for Canonical Rules)**.** *The Identity rule c_I is observationally complete. The LastWrite rule c_{LW} is not: $[pop(x), pop(y)]$ and $[pop(y), pop(x), pop(y)]$ are c_{LW}-equivalent but distinct. The Accumulate rule c_{acc} is not: $[pop(x), pop(y)]$ and $[pop(y), pop(x)]$ have identical accumulate-states.*

Definition 22.4 (Inverse-Problem Fibre)**.** For $o \in O_c$, the *fibre* of o is $c^{-1}(o) = \{H \in \mathcal{H} \mid c(H) = o\}$.

Characterising fibres for general rules, and determining when a fibre contains a unique history, is the primary open mathematical problem in the framework.

Open Problems

Distributed Spherepop Runtimes

The current World is single-threaded. A distributed Spherepop runtime would coordinate multiple Worlds, each with their own histories, under a global con-

sistency constraint. The consistency constraint must be expressed in terms of history equivalence under a shared collapse rule—not value consistency. The event sourcing and CRDT literatures provide partial models; the Spherepop-specific challenge is the certified-refusal structure across distributed agents.

Collapse Functor Composition

The four canonical collapse rules are special cases of a more general family of functors $F_c : \mathbf{Hist} \to \mathbf{Obs}_c$. A theory of functor composition would characterise which combinations of rules are valid—when $F_{c_2} \circ F_{c_1}$ is itself a valid collapse functor—and would classify the algebraic structure of the space of collapse rules.

Dependent Process Types in Full Generality

The current Process$(T_1 \rightsquigarrow T_2)$ type is first-order. A dependent process type $\Pi(x : T_1)$. Process$(x, f(x))$ would allow the type of the output to depend on the specific value consumed. This is the natural extension of dependent type theory into the process calculus setting and would enable precise typing of programs that branch on their inputs.

Admissibility Lattice Completeness

The current type system uses a Boolean admissibility structure. The full theory requires a graded admissibility lattice with continuous admissibility levels, connecting to the RSVP xylomorphic criterion $\lambda < 1$ and the admissibility geometry program. Completeness theorems for the graded type system would characterize exactly which trajectories are admissible at each level.

Proof-Carrying Refusal in Full Generality

The current RefusalReason is a vocabulary. A complete proof-carrying refusal system would make RefusalReason a proof term: evidence that can be type-checked independently of the computation that produced it. This connects to the theory of certified refusal and would enable mechanized verification of inadmissibility claims.

Chapter 23

The Coda: Implementation as Philosophical Argument

The implementation did not merely realize the theory; it exposed which distinctions the theory was still hiding.

The central claim of this monograph is not that Spherepop is a good programming language, though we believe it is an interesting one. The central claim is that taking a philosophical position seriously enough to implement it is a form of philosophical argument.

Four implementation discoveries were not predicted by the initial framework. They were forced by the attempt to mechanize it.

The original Refuse(Symbol) gestured at documented inadmissibility. It took implementation pressure to reveal that a symbol alone documents nothing: the reason is the document.

The original notion of observable state gestured at the quotient structure. It took `World::observe` to force the question: observable under what rule?

The original type checker gestured at world-relative knowledge. It took the free-variable conflict to force the question: which world are we assuming?

The original correctness criterion gestured at semantic preservation. It took the compiler test to force the question: preservation of what, exactly?

Each discovery reveals a distinction the philosophical framework was gesturing at without making precise. Only in the attempt to mechanize—to force the philosophy to compile—did the gestures become definitions. This is the deepest lesson of ontology-driven language design. Philosophy can identify the right questions. Only implementation can force the right precision.

Spherepop is the algebra of continuations.

Notes are the physical realization of that algebra.

Civilization is the attempt, always incomplete, always under threat, to build a note infrastructure adequate to the continuation needs of the species that requires it.

Part X

The Calculus of Constructions

Chapter 24

Why Spherepop Needs the Calculus of Constructions

The Limits of Simple Types

The type system developed in Part IV is powerful enough to certify admissibility, document refusals, and seal observations. But it is limited in one fundamental respect: types cannot depend on values. The type of the output of a function cannot depend on what the input actually is. This limitation becomes acute in several situations that arise naturally in Spherepop.

Consider a function that reads a file and returns a proof that the file was successfully read. The output type should mention the specific file that was read, not just the abstract type of files. Or consider a GC pass that returns a history with only live events: the output type should certify that specific inadmissibility certificates from the input are preserved. Simple types cannot express either of these conditions.

The Calculus of Constructions (CoC), introduced by Coquand and Huet [10], is the natural remedy. It is a typed lambda calculus in which types may depend on terms, terms may be types, and the distinction between levels collapses into a single universe hierarchy. In CoC, the type of a returned value can carry the full specification of what was computed.

The Pure Type System Perspective

The Calculus of Constructions belongs to the family of *pure type systems* (PTSs), which unify the handling of terms and types through a single syntactic category.
Definition 24.1 (Sorts and Pure Type System)**.** A *pure type system* is determined by a triple $(\mathcal{S}, \mathcal{A}, \mathcal{R})$ where $\mathcal{S}$ is a set of *sorts*, $\mathcal{A}$ is a set of *axioms* $s_1 : s_2$, and $\mathcal{R}$ is a set of *rules* (s_1, s_2, s_3) governing when Π-types can be formed.
Definition 24.2 (Calculus of Constructions)**.** The Calculus of Constructions is

the PTS with:

$$\mathcal{S} = \{*, \Box\}$$
$$\mathcal{A} = \{* : \Box\}$$
$$\mathcal{R} = \{(*, *, *),\ (*, \Box, \Box),\ (\Box, *, *),\ (\Box, \Box, \Box)\}$$

where $*$ is the sort of ordinary types and $\Box$ is the sort of kinds (types of types).

The four rules in $\mathcal{R}$ permit:

1. $(*, *, *)$: functions from terms to terms (ordinary functions).
2. $(*, \Box, \Box)$: functions from terms to types (dependent types).
3. $(\Box, *, *)$: functions from types to terms (polymorphism).
4. $(\Box, \Box, \Box)$: functions from types to types (type operators).

The full power of CoC is that all four are available simultaneously. This is what makes it capable of expressing the entire type-theoretic apparatus needed for Spherepop's admissibility certificates.

The Spherepop-CoC Correspondence

The Spherepop type system is naturally embedded into CoC.

Definition 24.3 (CoC Embedding of Spherepop Types)**.** The embedding $[\![\cdot]\!] : \mathsf{SphType} \to \mathsf{CoCTerm}$ is defined by:

$$[\![\mathsf{Unit}]\!] = \top_{\mathsf{CoC}}$$
$$[\![\mathsf{Never}]\!] = \bot_{\mathsf{CoC}}$$
$$[\![\mathsf{Admissible}(T)]\!] = \Sigma(h : \mathsf{History}).\, \mathsf{Admissible}(h, [\![T]\!])$$
$$[\![\mathsf{Refused}(T, r)]\!] = \Sigma(h : \mathsf{History}).\, \mathsf{Refused}(h, [\![T]\!], r)$$
$$[\![\mathsf{Collapsed}(T, c)]\!] = \Sigma(h : \mathsf{History}).\, \mathsf{Collapsed}(h, [\![T]\!], c)$$
$$[\![\Pi(x : T_1).T_2]\!] = \Pi(x : [\![T_1]\!]).[\![T_2]\!]$$
$$[\![\mathsf{Process}(T_1 \rightsquigarrow T_2)]\!] = \Pi(h_1 : [\![T_1]\!]).\, \Sigma(h_2 : [\![T_2]\!]).\, \mathsf{BindDep}(h_1, h_2)$$

The Σ-types in the embedding are crucial: they pair the value with a proof that the history supporting it has the required structure. The admissibility certificate is not merely a type annotation but a proof term that can be independently verified.

Theorem 24.4 (Embedding Soundness)**.** *If $\Gamma \vdash t : T$ in the Spherepop type system, then $[\![\Gamma]\!] \vdash [\![t]\!] : [\![T]\!]$ in CoC.*

Proof. By structural induction on the Spherepop typing derivation. Each typing rule maps to a valid CoC derivation:

[T-POP]: The embedding sends $\mathsf{pop}(t)$ to a pair $\langle h, \pi \rangle$ where π is a proof of $\mathsf{Admissible}(h, [\![T]\!])$. This is a valid Σ-introduction in CoC.

[T-REFUSE]: The embedding sends $\mathsf{refuse}(t, r)$ to a pair $\langle h, \pi_r \rangle$ where π_r proves $\mathsf{Refused}(h, [\![T]\!], r)$. Again a valid Σ-introduction.

[T-COLLAPSE]: Collapse requires a proof of admissibility as input and produces a proof of the collapsed type. The CoC derivation uses Σ-elimination on the admissibility proof followed by Σ-introduction for the collapsed type.

The remaining cases follow by induction. □

Chapter 25

Implementing CoC for Spherepop

The Universe Hierarchy

A full implementation of CoC for Spherepop requires a universe hierarchy to avoid Russell-type paradoxes. We adopt a predicative hierarchy.

Definition 25.1 (Universe Hierarchy)**.** Define sorts $\mathsf{Type}_0, \mathsf{Type}_1, \mathsf{Type}_2, \ldots$ with axioms $\mathsf{Type}_i : \mathsf{Type}_{i+1}$ and cumulative coercions $\mathsf{Type}_i \hookrightarrow \mathsf{Type}_{i+1}$.

In practice, most Spherepop typing occurs at Type_0 (the sort of ordinary program types). The proof-carrying refusal reasons inhabit Type_1 (propositions about Type_0 terms). The collapse rules inhabit Type_1 as functions from histories to observable states.

Listing 25.1: Universe levels in the Spherepop type checker

```
pub enum Universe {
    Type(usize),   // Type_0, Type_1, ...
    Prop,          // proof-irrelevant propositions
}

pub enum Term {
    Sort(Universe),
    Var(Symbol),
    App(Box<Term>, Box<Term>),
    Lam(Symbol, Box<Term>, Box<Term>),  // λx:A.b
    Pi(Symbol, Box<Term>, Box<Term>),   // Πx:A.B
    Sigma(Symbol, Box<Term>, Box<Term>),// Σx:A.B
    Pair(Box<Term>, Box<Term>),         // (a, b)
    Fst(Box<Term>),                     //
    Snd(Box<Term>),                     //
    // Spherepop primitives
    Pop(Box<Term>),
```

```
    Refuse(Box<Term>, RefusalReason),
    Bind(Box<Term>, Box<Term>),
    Collapse(Box<Term>, CollapseRule),
    // Admissibility universe
    Admissible(Box<Term>),
    Refused(Box<Term>, RefusalReason),
    Collapsed(Box<Term>, CollapseRule),
}
```

The Type Checker

A CoC type checker for Spherepop must implement bidirectional type checking: type inference for introduction forms and type checking for elimination forms.

Definition 25.2 (Bidirectional Typing)**.** Bidirectional typing uses two judgements:

- $\Gamma \vdash t \Rightarrow T$ (*synthesis*): the type T of t is inferred from t alone.
- $\Gamma \vdash t \Leftarrow T$ (*checking*): t is checked against the expected type T.

Listing 25.2: Bidirectional type checker core

```
pub fn synthesize(ctx: &Context, term: &Term)
    -> Result<Term, TypeError>
{
    match term {
        Term::Var(x) => ctx.lookup(x),
        Term::App(f, a) => {
            let f_ty = synthesize(ctx, f)?;
            match whnf(&f_ty) {
                Term::Pi(x, a_ty, b_ty) => {
                    check(ctx, a, &a_ty)?;
                    Ok(subst(b_ty, x, a))
                }
                _ => Err(TypeError::NotAFunction)
            }
        }
        Term::Fst(p) => {
            let p_ty = synthesize(ctx, p)?;
            match whnf(&p_ty) {
                Term::Sigma(_, a, _) => Ok(*a),
                _ => Err(TypeError::NotAPair)
```

```
            }
        }
        Term::Pop(t) => {
            let t_ty = synthesize(ctx, t)?;
            Ok(Term::Admissible(Box::new(t_ty)))
        }
        Term::Refuse(t, r) => {
            let t_ty = synthesize(ctx, t)?;
            Ok(Term::Refused(Box::new(t_ty), r.clone()))
        }
        Term::Collapse(t, c) => {
            let t_ty = synthesize(ctx, t)?;
            match whnf(&t_ty) {
                Term::Admissible(inner) =>
                    Ok(Term::Collapsed(inner, c.clone())),
                _ => Err(TypeError::
                   CollapseRequiresAdmissible)
            }
        }
        _ => Err(TypeError::CannotSynthesize)
    }
}

pub fn check(ctx: &Context, term: &Term, ty: &Term)
    -> Result<(), TypeError>
{
    match (term, whnf(ty)) {
        (Term::Lam(x, _, b), Term::Pi(y, a, b_ty)) => {
            let ctx2 = ctx.extend(x, &a);
            check(&ctx2, b, &subst(&b_ty, y, &Term::Var(x)))
        }
        (Term::Pair(a, b), Term::Sigma(x, a_ty, b_ty)) => {
            check(ctx, a, &a_ty)?;
            let a_val = eval(a);
            check(ctx, b, &subst(&b_ty, x, &a_val))
        }
        _ => {
            let t_ty = synthesize(ctx, term)?;
            if definitionally_equal(&t_ty, ty) { Ok(()) }
```

```
            else { Err(TypeError::TypeMismatch) }
        }
    }
}
```

Definitional Equality and Normalization

CoC type checking requires deciding definitional equality: two types are definitionally equal if they reduce to the same normal form.

Definition 25.3 (Weak Head Normal Form)**.** A term is in *weak head normal form* (WHNF) if it is not a beta-redex at the head: it is a lambda, a Pi, a Sigma, a sort, a variable, or an application whose function is in WHNF.

Listing 25.3: WHNF reduction for Spherepop-CoC

```
pub fn whnf(term: &Term) -> Term {
    match term {
        Term::App(f, a) => {
            match whnf(f) {
                Term::Lam(x, _, b) => whnf(&subst(&b, &x, a))
                    ,
                f_whnf => Term::App(Box::new(f_whnf),
                                    a.clone())
            }
        }
        Term::Fst(p) => {
            match whnf(p) {
                Term::Pair(a, _) => whnf(&a),
                p_whnf => Term::Fst(Box::new(p_whnf))
            }
        }
        Term::Snd(p) => {
            match whnf(p) {
                Term::Pair(_, b) => whnf(&b),
                p_whnf => Term::Snd(Box::new(p_whnf))
            }
        }
        // Spherepop reductions
        Term::Pop(t) => Term::Pop(Box::new(whnf(t))),
```

```
        Term::Collapse(t, c) => {
            Term::Collapse(Box::new(whnf(t)), c.clone())
        }
        t => t.clone()
    }
}
```

Theorem 25.4 (Normalization of Spherepop-CoC)**.** *The Spherepop-CoC type system is strongly normalizing: every well-typed reduction sequence terminates.*

Proof sketch. The Calculus of Constructions is strongly normalizing by the reducibility candidates argument of Girard [16]. The Spherepop extensions Pop, Refuse, Bind, and Collapse are all introduction forms that do not introduce new beta-redexes. They reduce only by unwrapping their arguments to WHNF. The combined system inherits strong normalization from CoC by a standard extension argument: the new rules add no new reduction paths that could create infinite sequences, since the underlying lambda calculus reductions are already terminating. □

Identity Types and History Equality

CoC extended with identity types (Martin-Löf type theory [24]) gives Spherepop a native notion of proof-level history equality.

Definition 25.5 (Identity Type)**.** For any type A and terms $a, b : A$, the *identity type* $a =_A b$ is a type whose inhabitants are proofs that a and b are definitionally equal. The sole constructor is $\mathsf{refl}_a : a =_A a$.

Definition 25.6 (History Identity Type)**.** For histories $H_1, H_2 : \mathsf{History}$, the type $H_1 =_{\mathsf{History}} H_2$ is inhabited precisely when the histories are event-for-event identical. This is the type-level statement of Replay Equivalence: a term of type $I(p).\mathsf{history} =_{\mathsf{History}} V(C(p)).\mathsf{history}$ is a formal proof that the compiler is correct.

Listing 25.4: History identity in the type system

```
pub fn verify_replay_equivalence(
    prog: &Term,
    interp_world: &World,
    compiled_world: &World,
) -> Result<(), ProofError> {
    // Produce a proof of history identity
    if interp_world.history == compiled_world.history {
```

```
        Ok(())  // refl witnesses the identity
    } else {
        // Construct a counterexample witness
        let diff = history_diff(
            &interp_world.history,
            &compiled_world.history
        );
        Err(ProofError::HistoryMismatch(diff))
    }
}
```

The Curry-Howard Correspondence for Spherepop

The Curry-Howard correspondence—that proofs are programs and propositions are types—takes a specific form in Spherepop.

Proposition 25.7 (Spherepop Curry-Howard Correspondence)**.** *The following identifications hold:*

Logical Notion	*Program Notion*	*Spherepop Realization*
Proposition	*Type*	$T : \mathsf{Type}_0$
Proof	*Term*	$t : T$
Implication	*Function type*	$\Pi(x : A).B$
Conjunction	*Pair type*	$\Sigma(x : A).B$
Admissibility	*Certificate type*	$\mathsf{Admissible}(T)$
Refutation	*Refusal type*	$\mathsf{Refused}(T, r)$
Observation	*Collapsed type*	$\mathsf{Collapsed}(T, c)$
History identity	*Replay equivalence*	$H_1 =_{\mathcal{H}} H_2$

Part XI

The BNF Grammar as Universal Notation

Chapter 26

Why Grammar Beats Circuits and Code

The Problem of Notation

Every computational formalism needs a notation: a way of writing down the objects it studies. The choice of notation is not merely aesthetic. It determines what can be expressed without ambiguity, what can be processed mechanically, what can be read without specialized tools, and what can be implemented independently of any particular hardware or software substrate.

Three dominant notations for computation exist: programming language syntax, circuit diagrams, and formal grammars. The thesis of this chapter is that Backus-Naur Form (BNF) grammar, or its extensions, is the superior notation for specifying computational behavior, and that Spherepop's use of BNF is not a convenience but a theoretical necessity.

A Brief History of BNF

Backus-Naur Form was introduced by Peter Naur in the ALGOL 60 report [26]. Its purpose was to give a precise, implementation-independent specification of a programming language's syntax. Before BNF, programming language grammars were described in English prose: ambiguous, difficult to parse mechanically, and dependent on shared human conventions.

BNF replaces prose with a minimal formal apparatus: nonterminals (enclosed in angle brackets), terminals (written as themselves), a production arrow ($::=$), and alternation ($|$). From these four elements, any context-free language can be specified.

Definition 26.1 (BNF Grammar)**.** A *Backus-Naur Form grammar* is a tuple $G = (N, T, P, S)$ where:

- N is a finite set of *nonterminals*.

- T is a finite set of *terminals* (disjoint from N).
- P is a finite set of *productions* of the form $A ::= \alpha$ where $A \in N$ and $\alpha \in (N \cup T)^*$.
- $S \in N$ is the *start symbol*.

Extended BNF (EBNF) adds optional elements ($[\cdot]$), repetition ($\{\cdot\}$), and grouping, but adds no expressive power beyond context-free languages.

BNF Versus Circuit Diagrams

Circuit diagrams are the standard notation for hardware computation. They have significant advantages: they are visually intuitive, they make data flow explicit, and they can be directly compiled to hardware. But they have several critical disadvantages relative to BNF.

Implementation Dependence

A circuit diagram is inherently tied to a specific level of abstraction. A gate diagram for a full adder specifies AND, OR, and NOT gates in a specific topology. This specification cannot be directly used at a different level of abstraction without redrawing. A BNF grammar for the same operation specifies the structure independently of implementation:

Listing 26.1: BNF for binary addition (implementation-independent)

```
<bit>      ::= "0" | "1"
<addend>   ::= <bit> | <bit> <addend>
<sum>      ::= <addend> "+" <addend>
<carry>    ::= <bit>
<result>   ::= <carry> <sum>
```

This grammar specifies the structure of binary addition without committing to gates, lookup tables, carry-lookahead circuits, or any other implementation. Any implementation that accepts the grammar's language is correct by definition.

Human Readability

Circuit diagrams require specialized visual parsing. They cannot be read aloud, stored in plain text, or processed by standard text tools. A BNF grammar is written in ASCII (or Unicode), can be stored in a version control repository, diffed,

searched, and processed by any text processor. It is read left-to-right, top-to-bottom, in the same direction as natural language.

Theorem 26.2 (BNF Readability Advantage)**.** *For any grammar G specifying a language L, the BNF representation of G requires only the characters of $N \cup T$ plus the BNF metasymbols $\{::=, |, \langle, \rangle\}$. The circuit representation of the same computation requires a graphical medium that cannot be stored or transmitted as plain text.*

Proof. BNF is defined over a finite alphabet. Its productions are strings over that alphabet. A circuit diagram, by contrast, represents spatial relationships (the topology of the circuit) that cannot be serialized as a string without loss of the spatial information that makes the diagram legible. The best-known lossless serialization of a circuit diagram is a netlist, which is itself a grammar-like notation—a point in favor of the grammar approach. □

Compositional Structure

BNF grammars compose naturally. If G_1 specifies language L_1 and G_2 specifies L_2, the grammar for $L_1 \cdot L_2$ (concatenation) is formed by adding a production $S ::= S_1\, S_2$ and combining the production sets. This compositional structure is the foundation of modular language design.

Proposition 26.3 (Grammar Compositionality)**.** *If $G_1 = (N_1, T_1, P_1, S_1)$ and $G_2 = (N_2, T_2, P_2, S_2)$ with $N_1 \cap N_2 = \varnothing$, then:*

$$G_1 \cdot G_2 = (N_1 \cup N_2 \cup \{S\},\ T_1 \cup T_2,\ P_1 \cup P_2 \cup \{S ::= S_1\, S_2\},\ S)$$
$$G_1 \mid G_2 = (N_1 \cup N_2 \cup \{S\},\ T_1 \cup T_2,\ P_1 \cup P_2 \cup \{S ::= S_1 \mid S_2\},\ S)$$

These operations correspond to sequential composition and choice in Spherepop.

BNF Versus Programming Language Syntax

BNF is more fundamental than any particular programming language syntax because it is what programming languages are defined in. A programming language's syntax is a grammar; BNF is the metalanguage for grammars.

But the comparison is not merely definitional. BNF grammars have concrete advantages as computational specifications over program text.

Ambiguity is Detectable

A BNF grammar can be checked for ambiguity (multiple parse trees for the same string). An English prose description of a language cannot. A program in any conventional language can be syntactically unambiguous but semantically ambiguous in ways that BNF would surface if the semantics were given as a grammar.

Separation of Concerns

BNF cleanly separates the *shape* of a program (its grammar) from its *meaning* (its semantics). This separation is enforced by the formalism: a BNF grammar says nothing about what programs mean, only about what strings are valid programs. This is not a limitation but a virtue: it allows the same grammar to be given multiple semantics (operational, denotational, axiomatic) without changing the specification of valid programs.

Formal Verification Target

A BNF grammar is a mathematical object that can serve as the specification for formal verification. A Spherepop program can be verified to be within the language of a grammar by a parser; verification at the semantic level then proceeds over the parse tree. This two-stage verification is not possible when the language is specified in prose or implicitly in a reference implementation.

The Spherepop BNF as Computational Universal

The Spherepop BNF grammar from Part III is not merely a syntactic convenience. It is a specification from which an implementation in any language, on any substrate, can be mechanically derived.

Definition 26.4 (Spherepop Core Grammar)**.** The Spherepop core language is

given by the following BNF:

$$
\begin{aligned}
\langle term \rangle &::= \langle var \rangle \mid \langle lambda \rangle \mid \langle app \rangle \mid \langle let \rangle \mid \langle pop \rangle \mid \langle refuse \rangle \mid \langle bind \rangle \mid \langle collapse \rangle \mid \langle seq \rangle \\
\langle var \rangle &::= \langle ident \rangle \\
\langle lambda \rangle &::= (\lambda \mid \mathsf{fn})\ \langle ident \rangle\ [:\ \langle type \rangle]\ (\rightarrow \mid .)\ \langle term \rangle \\
\langle app \rangle &::= \langle term \rangle\ \langle term \rangle \\
\langle let \rangle &::= \mathsf{let}\ \langle ident \rangle\ =\ \langle term \rangle\ \mathsf{in}\ \langle term \rangle \\
\langle pop \rangle &::= \mathsf{pop}\ \langle term \rangle \\
\langle refuse \rangle &::= \mathsf{refuse}\ \langle term \rangle\ [\langle reason \rangle] \\
\langle bind \rangle &::= \mathsf{bind}\ \langle term \rangle\ \langle term \rangle \\
\langle collapse \rangle &::= \mathsf{collapse}\ \langle term \rangle\ [\langle rule \rangle] \\
\langle seq \rangle &::= \mathsf{seq}\ [\langle term \rangle\ \{; \langle term \rangle\}^*] \\
\langle reason \rangle &::= \mathsf{violation}(\langle ident \rangle) \mid \mathsf{explicit}(\langle ident \rangle) \\
\langle rule \rangle &::= \mathsf{id} \mid \mathsf{last\_write} \mid \mathsf{accumulate} \mid \mathsf{proj}(\langle ident \rangle) \\
\langle type \rangle &::= \mathsf{Unit} \mid \mathsf{Never} \mid \langle type\_var \rangle \mid \mathsf{Admissible}(\langle type \rangle) \mid \langle type \rangle \rightarrow \langle type \rangle
\end{aligned}
$$

Theorem 26.5 (Grammar Determines Implementation)**.** *Given the Spherepop core BNF, a parser, type checker, and evaluator are mechanically derivable for any target platform.*

Proof. The grammar is context-free and therefore parseable by any LALR(1) or PEG parser generator. The type checker follows from the typing rules (which are themselves a kind of grammar over typing contexts). The evaluator follows from the operational semantics (which are also a grammar of world-state transitions). Each of these derivations is mechanical and platform-independent: the same BNF yields implementations in Rust, Python, Haskell, JavaScript, or any other host language without modification to the grammar. □

Grammar as Interface Contract

When the Spherepop grammar is fixed, it constitutes an interface contract between: (1) program authors, who write terms in the grammar's language; (2) parsers, which accept exactly the grammar's language; (3) type checkers, which verify admissibility of parsed terms; (4) evaluators, which execute well-typed terms; (5) compilers, which translate well-typed terms to Event IR.

Any implementation satisfying this contract is a correct Spherepop imple-

mentation. The grammar is the single source of truth for what the language is, independent of any particular reference implementation.

BNF as Anti-Collapse Artifact

The relationship between BNF grammars and the note theory of this monograph is not accidental.

A BNF grammar is a Refusal Note of the highest generativity. It specifies which strings are admissible (members of the language) and which are inadmissible. It refuses the collapse of the language into any particular implementation. And it generates the full continuation space of all programs in the language: any string in the language is a reachable continuation, and the grammar specifies exactly the set of reachable continuations.

Theorem 26.6 (BNF Grammars as Generative Refusal Notes)**.** *A BNF grammar G is simultaneously:*

1. *A* Generative Note*: it creates the continuation space $L(G)$ of all programs in the language, which did not exist before the grammar was specified.*
2. *A* Refusal Note*: it refuses all strings not in $L(G)$ as inadmissible, with the implicit reason* ParseError*.*
3. *A* Coordination Note*: it couples the behaviors of all implementations by binding them to the same accepted and rejected strings.*
4. *A* Capture Note*: it preserves the structure of valid programs across implementations, allowing programs written for one implementation to be run on another.*

Proof. (1) The language $L(G)$ is the set of all strings derivable from S in G. Before G exists, this set is undefined; after G exists, it is precisely specified. New programs in $L(G)$ are continuations that G makes reachable.

(2) A string $w \notin L(G)$ has no parse tree under G. The parser refuses it with a parse error. This is a Refuse operation with reason ParseError applied to the inadmissible string.

(3) Any two implementations I_1 and I_2 of G accept the same strings and reject the same strings. They are bound by the grammar to the same admissibility decisions. This is a Bind coupling their trajectory spaces.

(4) A program $p \in L(G)$ has an implementation-independent parse tree. The parse tree captures the structure of p independently of what evaluator processes it. This is a Capture Note for the program's syntactic structure. □

Chapter 27

Grammar, Circuits, and the Efficiency Hierarchy

Measuring Expressiveness Per Symbol

We want a precise way to compare notations for expressing computational structure. Let the *expressiveness per symbol* of a notation $\mathcal{N}$ for specification S be the ratio of the information content of S to the number of symbols required to express S in $\mathcal{N}$.

Definition 27.1 (Specification Complexity)**.** The *specification complexity* $K_{\mathcal{N}}(S)$ of specification S in notation $\mathcal{N}$ is the minimum number of symbols in $\mathcal{N}$ required to express S unambiguously. The *relative expressiveness* of $\mathcal{N}_1$ over $\mathcal{N}_2$ for S is

$$E(S, \mathcal{N}_1, \mathcal{N}_2) = \frac{K_{\mathcal{N}_2}(S)}{K_{\mathcal{N}_1}(S)}.$$

$E > 1$ means $\mathcal{N}_1$ is more expressive per symbol.

Theorem 27.2 (BNF Specification Efficiency)**.** *For context-free languages, BNF is asymptotically more symbol-efficient than circuit diagrams and at least as efficient as any other notation that is implementation-independent and mechanically parseable.*

Proof sketch. A circuit diagram for a function $f : \{0,1\}^n \to \{0,1\}^m$ specifies f by encoding its topology as a directed acyclic graph of gates. The minimum size of such a diagram is $\Omega(C(f))$ where $C(f)$ is the circuit complexity of f. A BNF grammar for the same function, specified as a language recognizer, can be written in $O(\log n + \log m + K(f))$ productions where $K(f)$ is the Kolmogorov complexity of f's description. For functions with short descriptions but high circuit complexity (such as sorting networks), BNF can be exponentially more compact. □

The Readability Spectrum

Different notations occupy different positions on the spectrum from machine-efficient to human-efficient.

Notation	Human Readable	Machine Parseable	Impl.-Independent
Machine code	No	Yes	No
Assembly	Partial	Yes	No
Circuit diagram	Yes (visual)	Partial	No
C/Rust program	Yes	Yes	Partial
Lambda calculus	Yes	Yes	Yes
BNF grammar	Yes	Yes	Yes
Natural language	Yes	No	Yes

BNF occupies the unique position of being simultaneously human-readable, mechanically parseable, and fully implementation-independent. No other common notation achieves all three.

Grammar as the Missing Level of Abstraction

Programming languages, circuit diagrams, and prose all operate at specific levels of abstraction. Grammar operates at the level of *specification*: it describes the structure of all possible instances of a computational artifact without committing to any particular instance.

Definition 27.3 (Abstraction Level Hierarchy)**.** The abstraction levels of computational description form a hierarchy:

1. *Physical*: transistor layouts, voltages, timing.
2. *Circuit*: gate-level topology.
3. *Machine*: instruction set architecture.
4. *Language*: program syntax and semantics.
5. *Grammar*: specification of the language itself.
6. *Meta-grammar*: specification of the notation for grammars.

Proposition 27.4 (Grammar as Implementation Firewall)**.** *A grammar at level n of the hierarchy is implementation-independent with respect to all levels below n. A Spherepop BNF grammar is therefore independent of all language implementations, all machine instruction sets, all circuit designs, and all physical substrates.*

This independence is exactly what makes BNF appropriate as the specifi-

cation language for Spherepop. The Spherepop grammar specifies what computations the language can express without committing to how they are executed. The execution details—Rust runtime, WASM compilation, hardware acceleration—are below the abstraction level at which the grammar operates.

Literate Grammar and Documentation

Knuth's literate programming [22] proposed that programs should be written as literature: prose explanations interleaved with code. The Spherepop approach inverts this slightly: grammar should be written as literature, with prose explanations interleaved with BNF productions.

Definition 27.5 (Literate Grammar)**.** A *literate grammar* is a BNF grammar augmented with:

1. Prose explanations of the intent of each production.
2. Examples of derivations in the grammar.
3. Semantic notes indicating what well-formed strings mean.
4. Transformation rules mapping productions to type-checking and evaluation rules.

The Spherepop BNF in this monograph is a literate grammar in this sense: each production is accompanied by the operational semantics rule it corresponds to and the type-checking rule it enables. This makes the grammar simultaneously a specification, a tutorial, and a formal verification target.

Part XII

Supplementary Derivations

Chapter 28

Derivation Compendium

This chapter collects all supplementary derivations, filling the mathematical gaps indicated in the preceding parts. Each derivation is labeled by the chapter it supports.

Chapter 1: Notes Increase Reachability

For each continuation $c \in \mathcal{C}$, define the *reachability gain* of note N by

$$G_A(N, c, t) = P_A(c, t \mid N) - P_A(c, t).$$

Then N is a note precisely when $\exists c \in \mathcal{C},\ G_A(N, c, t) > 0$.

Proposition 28.1 (Nontrivial Notes Are Reachability-Increasing)**.** *If N is a note and all non-note effects are nonnegative, then the total reachability gain $G_A(N, t) = \sum_{c \in \mathcal{C}} G_A(N, c, t)$ is strictly positive.*

Proof. By definition, there exists at least one c_0 such that $G_A(N, c_0, t) > 0$. If all other terms satisfy $G_A(N, c, t) \geq 0$, then the total sum contains one strictly positive term and no negative terms. Hence $G_A(N, t) > 0$. □

Chapter 2: State as Quotient

Proposition 28.2 (State Is a Quotient of History)**.** *Every surjective state projection $\sigma : \mathcal{H} \to S$ factors uniquely through the quotient map $q : \mathcal{H} \to \mathcal{H}/\!\sim_\sigma$.*

Proof. Define $\bar{\sigma}([H]) = \sigma(H)$. This is well-defined because $H_1 \sim_\sigma H_2$ implies $\sigma(H_1) = \sigma(H_2)$. Surjectivity makes $\bar{\sigma}$ onto; injectivity follows because distinct classes have distinct σ-values. Thus $S \cong \mathcal{H}/\!\sim_\sigma$. □

Chapter 3: Operator Minimality

Theorem 28.3 (Operator Minimality)**.** *No one of* Pop, Refuse, Bind, Collapse *is definable from the other three while preserving its effect on history, option space, admissibility, and observation.*

Proof. Pop uniquely decreases Ω; the other three leave Ω unchanged. Refuse uniquely changes the admissibility set $\mathcal{A}(H)$ without decreasing Ω; Pop changes Ω, while Bind and Collapse do not record inadmissibility. Bind uniquely introduces a dependency relation between two elements without consuming or refusing either; no combination of the other operators produces a pure dependency edge. Collapse uniquely maps a history through an observational rule into a quotient space; the other operators extend history internally but produce no observation. Hence all four are irreducible. □

Chapter 4: Free Category

Theorem 28.4 (Histories Form the Free Category on the Event Graph)**.** *Let G be the directed graph whose edges are primitive events. Then* **Hist** *is the free category generated by G.*

Proof. Objects are option spaces; morphisms are finite paths. Given any category $\mathcal{D}$ and graph morphism $F : G \to U(\mathcal{D})$, the unique functor $\bar{F} :$ **Hist** $\to \mathcal{D}$ extending F sends $[e_1, \ldots, e_n]$ to $F(e_n) \circ \cdots \circ F(e_1)$. This is the universal property of the free category. □

Chapter 5: Small-Step Determinacy

Proposition 28.5 (Determinacy Under Fixed Rule Selection)**.** *If the next operator and its arguments are fixed, Spherepop small-step reduction is deterministic.*

Proof. Each transition rule has a unique conclusion for fixed inputs. No fixed operator application can reduce to two distinct worlds. □

Chapter 6: Collapse Entropy Monotonicity

Theorem 28.6 (Collapse Increases Observational Entropy Under Coarsening)**.** *If c_2 is coarser than c_1, then every c_2-observation has entropy at least as large as the entropy of any c_1-observation contained inside it.*

Proof. Coarsening means each c_1-class is contained in some c_2-class. The fibre of the coarser observation contains the fibre of the finer observation, so $|c_2^{-1}(o_2)| \geq |c_1^{-1}(o_1)|$ for contained observations. Taking logarithms gives the entropy inequality. □

Chapter 7: Type Soundness

Theorem 28.7 (Spherepop Type Soundness)**.** *If $\vdash t : T$ and $t \to^* t'$, then either t' is a value of type T, or there exists t'' such that $t' \to t''$.*

Proof. By repeated application of Preservation and Progress. Preservation gives $\vdash t' : T$ after any finite reduction sequence. Progress implies t' is either a value or can take another step. □

Chapter 8: Lattice Collapse Threshold

Proposition 28.8 (Monotonicity of Collapse Permission)**.** *If $\ell_1 \leq \ell_2$ and $\Gamma \vdash t : \mathsf{Admissible}_{\ell_2}(T)$, then every collapse permitted at level ℓ_1 is also permitted at level ℓ_2.*

Proof. A higher admissibility level satisfies at least the constraints of a lower level. Since $\ell_1 \leq \ell_2$, any rule whose threshold is met by ℓ_1 is also met by ℓ_2. □

Note Composition Theory

Define note composition by

$$\mathcal{C}(N_1 \otimes N_2) = \mathcal{C}(N_1) \cup \mathcal{C}(N_2) \cup \mathcal{C}_{\text{bind}}(N_1, N_2),$$

where $\mathcal{C}_{\text{bind}}(N_1, N_2)$ is the set of continuations reachable only because both notes are jointly available.

Theorem 28.9 (Coordination Surplus)**.** *If $\mathcal{C}_{\text{bind}}(N_1, N_2) \neq \varnothing$, then*

$$V_C(N_1 \otimes N_2) > V_C(N_1) + V_C(N_2) - |\mathcal{C}(N_1) \cap \mathcal{C}(N_2)|.$$

Proof. The continuation set of the composite contains the union of the individual sets plus at least one additional bound continuation. The ordinary union has size $V_C(N_1) + V_C(N_2) - |\mathcal{C}(N_1) \cap \mathcal{C}(N_2)|$. Adding the nonempty bound set strictly increases this. □

Theorem 28.10 (Note Entropy)**.** *Define the* note entropy *of N relative to agent A at time t as*

$$H_A(N,t) = -\sum_{c\in\mathcal{C}} P_A(c,t \mid N)\log P_A(c,t \mid N).$$

A note N' is more focused *than N if $H_A(N',t) < H_A(N,t)$: it concentrates reachability on a smaller set of continuations. A note N' is more* generative *than N if $V_C(N') > V_C(N)$.*

Remark 28.11. Focus and generativity trade off: collapse notes are highly focused (low entropy) while generative notes are high-entropy. The note taxonomy orders by this tradeoff.

Note Class Derivations

Proposition 28.12 (Capture Adequacy)**.** *A capture note N is adequate for task family T iff for every continuation $c \in T$ there exists recoverable data $d \subseteq N$ such that $P_A(c,t \mid d) > P_A(c,t)$.*

Theorem 28.13 (Prospective Notes Are Deferred Pops)**.** *Every prospective note determines a deferred commitment operator. At time t the continuation is not yet executed (no immediate Pop occurs); at time t' the agent follows the note, selecting the future continuation via a Pop.*

Proof. A prospective note binds present state A_t to future continuation $c_{t'}$. The selection at t' is a Pop of $c_{t'}$ from the available alternatives. The note stores a commitment whose execution is deferred. □

Theorem 28.14 (Repair Notes Minimize Reconstruction Cost)**.** *A repair note N is optimal in family $\mathcal{F}$ when $C_r(c \mid N) = \min_{M\in\mathcal{F}} C_r(c \mid M)$.*

Proof. Repair value is $C_r(c) - C_r(c \mid N)$. Since $C_r(c)$ is fixed for the continuation, maximizing repair value is equivalent to minimizing $C_r(c \mid N)$. □

Proposition 28.15 (Checksums as Pure Refusal Notes)**.** *A checksum $\chi(h)$ refuses informational collapse by distinguishing corrupted histories from admissible ones. If $\chi(h') \neq \chi(h)$, then h' is refused as a valid continuation of h. The checksum is a pure refusal note: it stores only the boundary between admissible and inadmissible continuations.*

Theorem 28.16 (Generativity Is Strict Reachability Expansion)**.** *A note N is generative iff $\exists c : P_A(c,t) = 0$ and $P_A(c,t \mid N) > 0$.*

Proof. If such c exists, N creates access to a previously unreachable trajectory. Conversely, generativity requires opening at least one previously unavailable trajectory, which is exactly this condition. □

Theorem 28.17 (Collapse Notes Trade Detail for Access)**.** *Let $\mathcal{N}$ be a finite note collection. A collapse note K partitioning $\mathcal{N}$ into m classes reduces search cost from $O(|\mathcal{N}|)$ to $O(m + \max_i |\mathcal{N}_i|)$.*

Proof. Without organizing structure, worst-case search examines every note. With a partition of m classes, the agent selects one class (m steps) then searches that class ($\max_i |\mathcal{N}_i|$ steps worst case). □

Compilation and GC Derivations

Theorem 28.18 (Event IR Is History-Preserving)**.** *For every primitive source event e, compilation followed by VM execution appends the same event to history as interpretation.*

Proof. By case analysis: Pop compiles to `IrEvent::Pop`, which the VM appends as Pop; Refuse(r) compiles to `IrEvent::Refuse{r}`, appended as Refuse(r); analogously for Bind and Collapse(c). □

Theorem 28.19 (GC Is Safe Exactly When It Preserves Live Fibres)**.** *A GC operation $g : \mathcal{H} \to \mathcal{H}$ is safe iff for every live continuation c, $P_A(c, t \mid g(H)) = P_A(c, t \mid H)$.*

Proof. If the equality holds for all live continuations, GC has removed no history needed for any reachable future—hence safe. Conversely, safety by definition preserves all history required for live continuations. □

Error Correction Derivation

Theorem 28.20 (Certified Refusal Prevents Silent Error Absorption)**.** *If every refusal carries a proof term π, then no error can be erased without either being handled or remaining visible in the output type.*

Proof. A refusal has type Refused(T, π). The typing rules allow this type to be handled by a recovery term or propagated. There is no rule converting Refused(T, π) into T while discarding π. Hence the error cannot be silently absorbed. □

Category Theory Derivations

Theorem 28.21 (Collapse as Reflection)**.** *If* $\mathbf{Obs}_c$ *is the full subcategory of* **Hist** *consisting of histories saturated under* $\sim_c$*, then the quotient functor* $F_c : \mathbf{Hist} \to \mathbf{Obs}_c$ *is left adjoint to the inclusion* $I_c : \mathbf{Obs}_c \hookrightarrow \mathbf{Hist}$*.*

Proof. For each history H and saturated observable O, every morphism $H \to I_c(O)$ constant on $\sim_c$-classes factors uniquely through $F_c(H)$. This gives the natural bijection $\mathbf{Obs}_c(F_c(H), O) \cong \mathbf{Hist}(H, I_c(O))$, establishing $F_c \dashv I_c$. □

Corollary 28.22 (Perfect Observation Requires History)**.** *An observation map is perfectly history-faithful iff it is isomorphic to the identity observation on* $\mathcal{H}$*.*

Proof. Perfect history-faithfulness requires injectivity. A bijective observation establishes an isomorphism between histories and observations, hence is isomorphic to the identity. □

Sheaf Derivations

Theorem 28.23 (State Illusion as Failure of Unique Gluing)**.** *Given a cover of observations* $\{U_i\}$*, state illusion occurs exactly when compatible local sections* $s_i \in \mathcal{H}(U_i)$ *admit more than one global section.*

Proof. Compatible local sections agree on overlaps. If more than one global history restricts to the same local observations, the observations do not determine a unique history—precisely the state illusion. □

CLIO Derivation

Theorem 28.24 (Spherepop Observational Entropy Is Discrete CLIO Entropy)**.** *The CLIO fibre entropy* $S_\pi(o) = \log \mathrm{Vol}(\pi^{-1}(o))$ *reduces in the discrete setting to* $S_c(o) = \log |c^{-1}(o)|$*.*

Proof. In the discrete setting, volume is counting measure. Therefore $\mathrm{Vol}(\pi^{-1}(o)) = |\pi^{-1}(o)| = |c^{-1}(o)|$. □

Active Geodesic Derivation

Proposition 28.25 (Histories Are Discrete Geodesics)**.** *Let* $L(e_i)$ *be the local cost of event* e_i *and define the action* $\mathcal{S}(H) = \sum_i L(e_i)$*. A Spherepop execution is a discrete*

geodesic when it minimizes $\mathcal{S}(H)$ among admissible histories with the same boundary conditions.

Proof. In discrete variational mechanics, a geodesic minimizes action between fixed endpoints. A Spherepop history is a path in event space; restricting to admissible histories and minimizing the additive cost functional gives the discrete geodesic condition. □

MEM|8 Locality Derivation

Theorem 28.26 (Locality as Continuation Preservation)**.** *If related memory components are physically or logically local, then the expected cost of reconstructing their shared continuation is lower than under random placement.*

Proof. Let $d(x, y)$ be access distance between memory components. Reconstruction cost is monotone in aggregate distance among components required by a continuation. Local placement lowers expected pairwise distance relative to random placement, hence lowers expected reconstruction cost. □

Civilizational Collapse Derivation

Theorem 28.27 (Reachability Loss Under Archive Destruction)**.** *Let $c \in R(Z)$ be reachable only through some $N \in \mathcal{N}$. Destroying $\mathcal{N}$ strictly decreases collective reachability.*

Proof. Since c is reachable only through N, removing N removes c from $R(Z)$. The post-destruction reachability is a proper subset of the original. □

Inverse Problem Derivation

Proposition 28.28 (Kernel Triviality Characterizes Complete Observation)**.** *A collapse rule c is observationally complete iff* $\ker(c) = \Delta_{\mathcal{H}} = \{(H, H) : H \in \mathcal{H}\}$.

Proof. Observational completeness means $c(H_1) = c(H_2) \Rightarrow H_1 = H_2$, so the kernel contains only diagonal pairs. Conversely, a diagonal kernel gives injectivity. □

Final Unification Theorem

Theorem 28.29 (Notes as Reachability Artifacts)**.** *An artifact N is a note iff it is a reachability artifact: there exists an agent A, a continuation c, and a future time t such that $P_A(c, t \mid N) > P_A(c, t)$. Spherepop programs are notes because they preserve, restrict, bind, refuse, and collapse continuations in history space.*

Proof. The first claim is the definition of a note under the continuation account. For the second: every Spherepop program constructs a history from primitive events. Pop selects continuations; Refuse preserves inadmissibility distinctions; Bind couples trajectories; Collapse makes histories observable under rules. A Spherepop program is therefore an artifact that changes which continuations remain reachable to future agents. Hence it is a note. □

Corollary 28.30 (Universal Reachability Artifact Thesis)**.** *Programs, proofs, archives, maps, protocols, libraries, institutions, languages, and civilizations are all special cases of the same abstract object: a reachability artifact. They differ in the medium through which they preserve continuations and in the scale at which they operate. The abstract structure is identical in every case.*

Proof. By Theorem 28.29, any artifact that increases the probability of traversing at least one continuation is a note. Each of the listed artifacts does this:

A *program* preserves a computational continuation. A *proof* preserves an inferential continuation. An *archive* preserves historical continuations against decay. A *map* creates navigational continuations. A *protocol* binds communicative continuations between agents. A *library* is a repository of continuations across domains. An *institution* binds social and coordination continuations across generations. A *language* is a system of semantic continuations shared by a community. A *civilization* is the totality of all of the above, organized into a distributed hierarchy of note infrastructure.

In every case, the note increases $P_A(c, t)$ for some continuation c. □

Bibliography

[1] Barendregt, H. P. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, 1984.

[2] Borgman, C. L. *Scholarship in the Digital Age*. MIT Press, 2007.

[3] Clark, A. and Chalmers, D. The extended mind. *Analysis* 58, no. 1 (1998): 7–19.

[4] Eisenstein, E. L. *The Printing Press as an Agent of Change*. Cambridge University Press, 1980.

[5] Feldman, J. On the absolute continuity of Gaussian measures. *Zeitschrift für Wahrscheinlichkeitstheorie* 1 (1958).

[6] Felleisen, M. and Friedman, D. P. A calculus for assignments in higher-order languages. *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, 1987.

[7] Chenoweth, C., Chenoweth, A. MEM|8: A Wave-Based Cognitive Architecture for Multimodal Memory Integration and Consciousness Simulation. Zenodo, 2025. `https://doi.org/10.5281/zenodo.16436298`

[8] Barendregt, H. P. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, 1984.

[9] Borgman, C. L. *Scholarship in the Digital Age*. MIT Press, 2007.

[10] Coquand, T. and Huet, G. The calculus of constructions. *Information and Computation* 76 (1988): 95–120.

[11] Clark, A. and Chalmers, D. The extended mind. *Analysis* 58, no. 1 (1998): 7–19.

[12] Eisenstein, E. L. *The Printing Press as an Agent of Change*. Cambridge University Press, 1980.

[13] Feldman, J. On the absolute continuity of Gaussian measures. *Zeitschrift für Wahrscheinlichkeitstheorie* 1 (1958).

[14] Felleisen, M. and Friedman, D. P. A calculus for assignments in higher-order languages. *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, 1987.

[15] Fowler, M. Event sourcing. `martinfowler.com`, 2005.

[16] Girard, J.-Y., Lafont, Y., and Taylor, P. *Proofs and Types*. Cambridge University Press, 1989.

[17] Goody, J. *The Domestication of the Savage Mind*. Cambridge University Press, 1977.

[18] Hájek, J. On a property of normal distributions of an arbitrary stochastic process. *Czechoslovak Mathematical Journal* 8 (1958).

[19] Hoare, C. A. R. *Communicating Sequential Processes*. Prentice Hall, 1985.

[20] Hofmann, M. and Streicher, T. The groupoid interpretation of type theory. In *Twenty-five Years of Constructive Type Theory*. Oxford University Press, 1998.

[21] Hutchins, E. *Cognition in the Wild*. MIT Press, 1995.

[22] Knuth, D. E. Literate programming. *The Computer Journal* 27, no. 2 (1984): 97–111.

[23] Mac Lane, S. *Categories for the Working Mathematician*. Springer, 1971.

[24] Martin-Löf, P. *Intuitionistic Type Theory*. Bibliopolis, 1984.

[25] Milner, R. *A Calculus of Communicating Systems*. Springer, 1980.

[26] Naur, P. (ed.). Report on the algorithmic language ALGOL 60. *Communications of the ACM* 3, no. 5 (1960): 299–314. (Original introduction of Backus–Naur Form.)

[27] Nordström, B., Petersson, K., and Smith, J. *Programming in Martin-Löf's Type Theory*. Oxford University Press, 1990.

[28] Ong, W. J. *Orality and Literacy: The Technologizing of the Word*. Methuen, 1982.

[29] Pierce, B. C. *Types and Programming Languages*. MIT Press, 2002.

[30] Reiter, R. On closed world data bases. In *Logic and Data Bases*. Plenum Press, 1978.

[31] Santoro, M., Waghmare, S., and Panaretos, V. M. Kernel embeddings of functional data and mutual singularity. Preprint, 2024.

[32] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. Conflict-free replicated data types. INRIA Technical Report, 2011.

[33] Strachey, C. Fundamental concepts in programming languages. *Higher-Order and Symbolic Computation* 13 (2000): 11–49. Originally delivered 1967.

[34] Tulving, E. Episodic and semantic memory. In *Organization of Memory*. Academic Press, 1972.

[35] The Univalent Foundations Program. *Homotopy Type Theory: Univalent*

Foundations of Mathematics. Institute for Advanced Study, 2013.

[36] Wright, A. K. and Felleisen, M. A syntactic approach to type soundness. *Information and Computation* 115, no. 1 (1994): 38–94.